Low-Level Design Report

Papillon
Course: BIL-496
Date: 5 March 2026

This report describes the internal design of the Papillon cybersecurity platform, developed as a Senior Design Project.

1. Introduction

Cybersecurity threats are becoming increasingly sophisticated in our interconnected world. Individual users and small organisations often lack the tools or expertise to assess their own digital risk, detect phishing attempts, evaluate password strength, or monitor publicly disclosed vulnerabilities. Papillon aims to bridge this gap by providing a unified, accessible cybersecurity platform that puts professional-grade tools in the hands of SOC users.

The core purpose of Papillon is to offer an integrated suite of security capabilities: session-based authentication enhanced with Multi-Factor Authentication (MFA), symmetric and asymmetric cryptographic operations, real-time feed of publicly known CVE vulnerabilities from the NIST NVD, attack surface analysis through a collection of reconnaissance tools, Microsoft Outlook integration for phishing detection, and AI-driven models for password strength evaluation and network intrusion classification.

This report describes the low-level architecture and design of the Papillon platform. It covers object design trade-offs, engineering standards, package descriptions with UML-style diagrams, and complete class interface specifications for all server and client components.

1.1 Object Design Trade-offs

Asynchronous Processing: To avoid blocking the main web server, heavy tasks (such as parsing logs or running AI models) are offloaded to a message queue (RabbitMQ/Redis) and processed by a pool of Celery workers. To support real-time situational awareness, the system implements an event-driven notification mechanism; once a background worker completes a task or detects a critical threat, an alert is dispatched asynchronously to the frontend.

Subsystem decomposition:

- **Presentation Layer:**
  - **Dashboard UI:** Handles visualization of alerts, graphs, and reports.
  - **WebSocket Gateway:** Manages real-time data push to clients, serving as the core infrastructure for the system's live notification and event-driven alerting features.
- **Application/API Layer:**
  - **API Collector:** The entry point for external webhooks (e.g., raw IDS logs) and internal requests. It handles initial **log management**, validates incoming payloads, and seamlessly queues them to the message broker.
  - **Auth & User Management:** Handles login, Role-Based Access Control (RBAC) to distinguish between administrators and standard users, and user profiles.
- **Service/Processing Layer (Worker Pool):**
  - **Normalizer Worker:** A dedicated normalization layer that consumes raw JSON logs from disparate IDS vendors via the API Collector and converts them into the system's unified IDS alerting schema.
  - **Inference Service (ML Orchestrator):** Runs AI models for anomaly and phishing detection. It provides an internal endpoint (e.g., via a dedicated Django API / FastAPI) to load models and return classification scores.

    - **Sandbox Manager:** Provisions isolated VMs (e.g., via CALDERA) for malware analysis and attack simulation.
    - **Scan Service:** Orchestrates tools (Nmap, OpenSSL, encompassing all attack surface layer components) to scan a domain and return a structured JSON report of open ports and certificate status.
    - **Reporting Service:** Aggregates data from the database and renders HTML templates into downloadable PDF documents.
    - **CVE Fetcher:** Periodically pulls vulnerability data from external APIs (e.g., fetching the last 60 days of records from NIST NVD).
- **Data Layer:**
    - **Primary DB:** MySQL database for structured data. *(Note: The detailed database schema and entity-relationship models are provided in the Appendices).*
    - **Cache/Queue:** Redis and RabbitMQ are utilized for session management, task queuing, and orchestrating the management structure of the worker nodes.

1.1.1 Usability vs. Functionality

Papillon bridges the gap between complex Enterprise/SOC functionalities (such as IDS log parsing, CALDERA sandboxing, and RBAC) and accessibility for small-to-medium enterprise (SME) users. While the backend supports an advanced, event-driven threat intelligence architecture, the frontend UI is strictly streamlined to prevent overwhelming the user. For example, the Encryption page presents a single algorithm selector rather than exposing raw cryptographic parameters. This trade-off accepts some loss of fine-grained configuration in exchange for a highly intuitive, guided user experience.

1.1.2 Security vs. Performance

Strong security frequently comes at the cost of performance. Papillon uses AES-256-GCM for symmetric encryption and RSA-2048 with OAEP padding for asymmetric operations. Passwords are hashed using Django's PBKDF2+SHA256 with a high iteration count to resist brute-force attacks. While the event-driven architecture and message brokers (RabbitMQ) introduce slight serialization overhead, this trade-off is absolutely necessary to ensure the main API remains secure, stable, and performant during heavy ML inference or sandbox provisioning. Furthermore, MFA setup stores TOTP secrets in the server-side session until the first successful OTP verification, preventing incomplete setups from leaving orphaned secrets.

1.1.3 Efficiency vs. Accuracy

In designing the attack surface analysis module (Scan Service), a definitive architectural trade-off was made between execution latency (Efficiency) and the completeness of the reconnaissance data (Accuracy). A comprehensive security scan—such as a full 65,535 TCP port scan or a deep brute-force subdomain enumeration—is highly accurate but computationally expensive. Holding HTTP requests open for this duration is unviable and risks blocking Celery workers or causing UI timeouts. To guarantee a highly responsive user experience, the system deliberately restricts its search space (e.g., scanning only nine curated, high-risk ports). We explicitly accept a higher rate of false negatives in exchange for high-speed, on-demand situational awareness that returns actionable results within seconds.

1.1.4 Reliability vs. Compatibility

The backend is a Django application targeting Python 3.11 and MySQL 8.0. Supporting legacy database engines or older Python versions would increase maintenance complexity without meaningful benefit.

The reliance on a robust worker pool (Celery/RabbitMQ) guarantees high reliability for heavy tasks, trading off the simplicity of a single-process deployment for a more resilient, distributed control plane. HashiCorp Vault is used for secret management; in development, the application falls back gracefully to environment variables, but production deployments strictly enforce Vault availability at startup via the `VAULT_ENFORCE` flag to ensure absolute credential reliability.

1.2 Interface Documentation Guidelines

In this report, all classes, attributes, and methods follow the naming conventions of their respective language. Python classes use PascalCase; methods and attributes use snake_case. JavaScript/React components use PascalCase; functions and variables use camelCase. Class interface specifications are given in the following format:

class ClassName
Explanation of the class.
Attributes
`access_modifier  type  attribute_name`
Methods
`return_type  method_name( parameters )`
// Method explanation if necessary
Edge Cases and Error Handling
<method_name> → <error_condition> → <what the system returns/does>
Testability Notes
Unit: <what it tests> → <expected result> Integration: <which flow it tests> → <expected result>

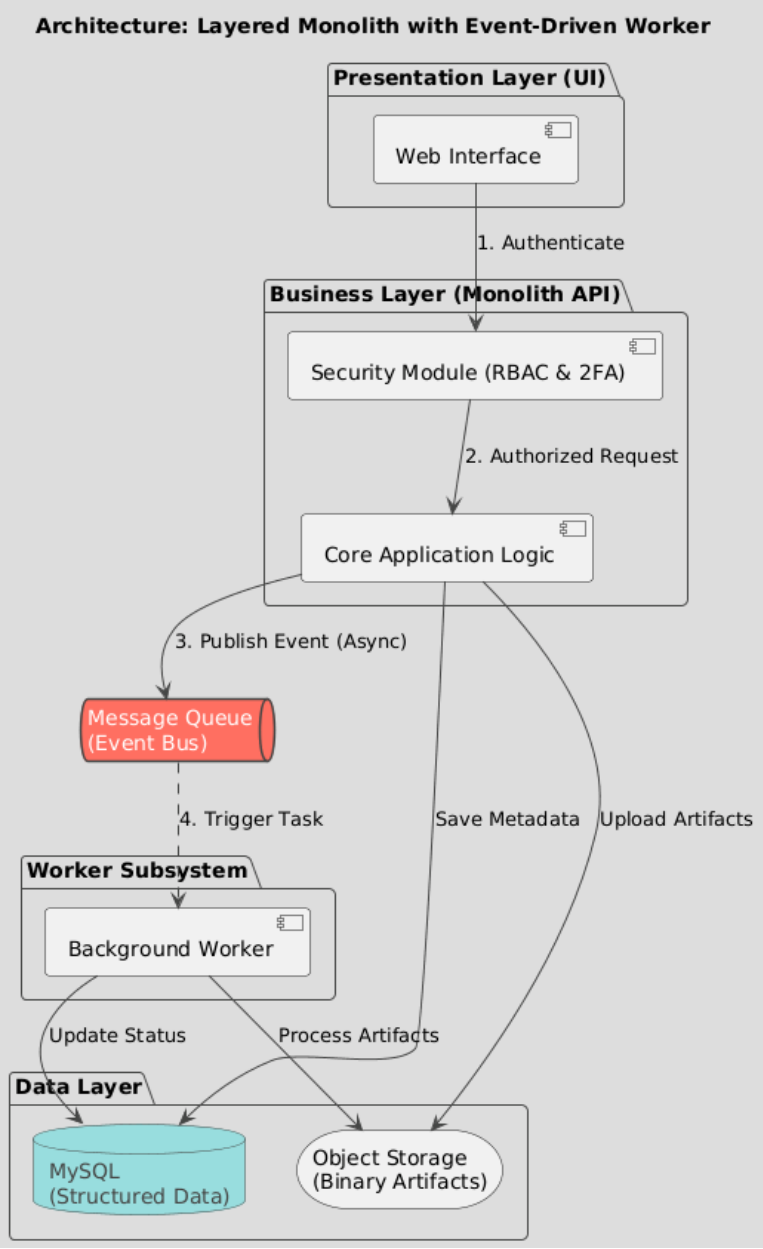
Attribute types are given in the language-idiomatic form (e.g., str, int, bool for Python; string, number, boolean for JavaScript). Where a type is a collection, it is written as List[Type] or Dict[KeyType, ValueType]. Optional attributes are marked with a [?] suffix.

All internal and external RESTful interfaces strictly adhere to the OpenAPI 3.0 specification. Communication between the client application and the backend services is performed exclusively using application/json payloads. Standard HTTP methods (GET, POST) are systematically mapped to their respective operations. To maintain consistency and predictability across the platform, all endpoints utilize standard HTTP status codes for response and error handling (e.g., 200 OK for success, 201 Created for resource creation, 400 Bad Request for validation errors, and 401 Unauthorized for authentication

failures). Detailed endpoint definitions, including request parameters and response schemas, are dynamically generated and available via Swagger UI within the development environment.

1.3 Engineering Standards

The UML guidelines are used in this report for class interface descriptions, package diagrams, and subsystem compositions[1]. Code style follows PEP 8 for Python, which is strictly enforced by 'black' and 'ruff'. To maintain a lightweight production environment, these linting and formatting tools are isolated as development dependencies and managed via a pyproject.toml configuration. JavaScript code adheres to the Airbnb ESLint ruleset[2].



1.4 Definitions, Acronyms, and Abbreviations

AI: Artificial Intelligence.

API: Application Programming Interface.

B2C: Business-to-Consumer.

CSRF: Cross-Site Request Forgery — a web attack type.

CTI: Cyber Threat Intelligence.

CVE: Common Vulnerabilities and Exposures — a public catalogue of disclosed security flaws.

CVSS: Common Vulnerability Scoring System — a standardized system used to assess and score the severity of security vulnerabilities.

DB: Database.

DNS: Domain Name System.

GDPR: General Data Protection Regulation.

HLD: High-Level Design.

HTML: HyperText Markup Language.

HTTP: Hypertext Transfer Protocol.

IDS: Intrusion Detection System.

IP: Internet Protocol.

IPv4: Internet Protocol version 4.

JSON: JavaScript Object Notation.

JWT: JSON Web Token.

KV: Key-Value secret engine in HashiCorp Vault.

LLD: Low-Level Design.

MFA: Multi-Factor Authentication.

ML: Machine Learning.

MVC: Model-View-Controller architecture pattern.

NVD: National Vulnerability Database.

OAEP: Optimal Asymmetric Encryption Padding.

ONNX: Open Neural Network Exchange format.

OTP: One-Time Password.

PDF: Portable Document Format.

PE: Portable Executable.

QR: Quick Response.

RBAC: Role-Based Access Control.

REST: Representational State Transfer.

SME: Small and Medium-sized Enterprises.

SOC: Security Operations Center.

TOTP: Time-based One-Time Password (RFC 6238).

UI: User Interface.

UML: Unified Modeling Language.

VM: Virtual Machine.

2. Packages

2.1 Server

The server is a Django project located at backend/papillon/. It is composed of five Django applications (users, crypto, cve, outlook, attack_surface), a core configuration package (papillon), and two standalone tool directories (attack_surface_analysis_tools, ai_models) that are invoked at runtime.

2.1.1 Model

Model
CustomUser
EncryptedData
OutlookAccount
↳ get_cipher()
Note: cve and attack_surface apps have no persisted Django models;
data is fetched from external APIs and returned directly to the client.

CustomUser: This class represents the main user entity of the platform. It stores credentials, session-relevant metadata, and MFA configuration. Passwords are stored as Django PBKDF2+SHA256 hashes. The MFA secret is a Base32 TOTP seed that is written to the database only after the user successfully verifies the first OTP code during MFA setup.

EncryptedData: This class records every cryptographic operation performed by a user. It supports symmetric encryption (AES-256-GCM), asymmetric encryption (RSA-2048), and one-way hash functions (MD5, SHA-1, SHA-256, SHA-512, Base64). The algorithm field determines which subset of the ciphertext, key, nonce, private_key, public_key, and hash_result fields are populated.

OutlookAccount: This class stores the Microsoft OAuth tokens that allow Papillon to read a user's Outlook inbox on their behalf. All four sensitive fields (_access_token, _refresh_token, _client_id, _client_secret) are encrypted at rest using Fernet symmetric encryption before being persisted to the

database. The get_cipher() helper retrieves the encryption key from the ENCRYPTION_KEY environment variable.

2.1.2 Controller

Controller
UserController (users/views.py)
register, login, logout, dashboard
mfa_setup, mfa_verify_setup, verify_mfa
mfa_disable, mfa_status
CryptoController (crypto/views.py)
encrypt_text, decrypt_text
CVEController (cve/views.py)
latest_cves
OutlookController (outlook/views.py)
save_client_id, authorize, callback
disconnect, get_outlook_status, get_latest_mail
AttackSurfaceController (attack_surface/views.py)
attack_surface_scan
VaultService (papillon/vault_service.py)
get_secret, write_secret, health_check

UserController: Handles all authentication and MFA lifecycle operations. Login implements a two-phase flow: if the user has MFA enabled, the first POST to /auth/login/ verifies credentials and stores a short-lived mfa_pending_user token in the session; a subsequent POST to /auth/mfa/verify/ supplies the TOTP code to complete authentication. Without MFA the session user_id is set immediately on successful credential verification.

CryptoController: Exposes encrypt_text and decrypt_text endpoints. Each algorithm branch handles key generation, performs the cryptographic operation using the Python cryptography library, persists an EncryptedData record, and returns the result. Hash functions (MD5, SHA-1, SHA-256, SHA-512) do not support decryption and return an error if the decrypt endpoint is called with those algorithms.

CVEController: Queries the NVD REST API v2.0 for CVEs published within the last 60 days. The number of results is controlled by the limit query parameter (clamped to 1-50). Severity and CVSS scores are extracted from the CVSS v3.1 metrics block, falling back to CVSS v2 if v3.1 is not available.

OutlookController: Implements a full Microsoft OAuth 2.0 authorization code flow. The user supplies their Azure application Client ID and Client Secret, which are stored temporarily in the server-side session. After the OAuth callback, the access and refresh tokens are encrypted and persisted in the OutlookAccount model. The get_latest_mail endpoint retrieves the most recent message from the user's inbox via the Microsoft Graph API.

AttackSurfaceController: Accepts a domain name, resolves it to an IP address, and then calls all eight reconnaissance tool modules in sequence. Each module is imported lazily so that missing optional dependencies (e.g., nmap) only affect that specific sub-scan rather than the entire request.

VaultService: A singleton helper that manages the HashiCorp Vault connection. It tries Vault first (KV v2), falls back to .env, and finally to the supplied default value. A secret cache avoids repeated Vault round-trips within a single process lifetime.

2.1.3 AI Modules

AI Modules (backend/ai_models/)
PasswordStrengthModule
import model + POST /predict
extract_features() + export model
NetworkIntrusionModule
import model + POST /predict
preprocessing artefacts + export model
MalwareDetectionModule
import model + POST /predict
extract_features() + export model
PhishingDetectionModule
ONNX Runtime inference (Singleton)
PhishingDetector.predict(email_text)

PasswordStrengthModule: Serves as a machine learning-based password analysis and classification module. Passwords are converted into numerical features based on metrics such as length, uppercase/lowercase presence, digit/special character counts, and entropy to measure theoretical information density. The classification architecture utilizes a Random Forest Classifier with 300 decision trees, trained on a dataset of approximately 300,000 samples with a 95% accuracy rate and serialized via joblib. Through the predict_strength endpoint, incoming POST requests analyze the input password and return a JSON response containing the prediction as "Weak" (0), "Normal" (1), or "Strong" (2).

NetworkIntrusionModule: IntrusionDetectionModule is a machine learning-based security component integrated into a Django web framework to analyze network traffic and identify potential cyber threats.

The module processes raw network data through a preprocessing pipeline that includes feature scaling with StandardScaler and categorical encoding with LabelEncoder to ensure data compatibility. Utilizing an XGBoost architecture, the system categorizes network activities as either "Normal" or as specific attack types such as DoS, DDoS, Web Attack, Port Scan. Using joblib to load the serialized XGBoost model and returning structured JSON responses for dynamic threat monitoring.

.

MalwareDetectionModule: MalwareDetectionModule is a machine learning-based security component integrated into a Django web framework, designed to perform static analysis on Portable Executable (PE) files. The module utilizes a specialized feature extraction pipeline that processes raw binary data into a Byte-Value Histogram and calculates Byte-Entropy levels to identify hidden patterns within the file structure. For the classification task, the system employs a Random Forest Classifier trained on these statistical distributions to distinguish between benign and malicious software. The backend is architected to receive file uploads directly through a Django view, where the input .exe is analyzed in real-time by the extraction engine. Using joblib to load the pre-trained model, the system performs instantaneous inference and returns a JSON response indicating the file's legitimacy, enabling automated threat triage without execution.

PhishingDetectionModule: An ONNX-format binary classifier loaded via the onnx runtime package. The PhishingDetector class follows the Singleton pattern so that the model is loaded into memory only once per process. The predict() method accepts raw email text and returns a result dictionary containing is_phishing (bool) and label ("PHISHING" or "SAFE").

2.1.4 Attack Surface Tools

Attack Surface Tools (backend/attack_surface_analysis_tools/)
dns_records.get_records(domain)
subdomain_finder.get_subdomains(domain)
whois_search.get_whois(domain)
ssl_scanner.scan_ssl_cert(domain)
port_scanner.scan_ports(ip)
email_harvester.harvest_emails(domain)
admin_panel_scanner.find_admin_panels(domain)
robots_parser.get_robots_txt(domain)
ip_address.get_ip(domain)

dns_records: Queries eight DNS record types (A, AAAA, NS, CNAME, MX, TXT, SOA, CAA) using the dnspython library.

subdomain_finder: Tests a wordlist of common subdomain prefixes using concurrent HTTP probes. Limited to MAX_PROBES = 100 candidates per scan.

whois_search: Returns structured WHOIS data for the domain using the python-whois library.

ssl_scanner: Performs a full TLS handshake analysis via sslyze, returning certificate details and a validity flag.

port_scanner: Probes nine high-risk ports (21, 22, 23, 80, 443, 445, 3306, 3389, 8080). Uses python-nmap if available, otherwise falls back to socket-based probing.

email_harvester: Scrapes the domain homepage and several common sub-pages (contact, about, team, support) for email addresses using a regex pattern.

admin_panel_scanner: Probes a list of 13 common admin URL paths using concurrent HEAD/GET requests.

robots_parser: Fetches and returns the plain text of the domain's robots.txt file.

ip_address: Resolves the domain to its primary IPv4 address using socket.getaddrinfo.

2.2 Client

2.2.1 View

View (frontend/src/)
LoginView (pages/Login.jsx)
RegisterView (pages/Register.jsx)
DashboardView (pages/Dashboard.jsx)
EncryptionView (pages/Encryption.jsx)
CVEListView (pages/CVEList.jsx)
AttackSurfaceView (pages/AttackSurfaceAnalysis.jsx)
MFASettingsView (pages/MFASettings.jsx)
OutlookCallbackView (pages/OutlookCallback.jsx)
ApiService (services/api.js)

LoginView: Manages a two-step login flow. In the first step the user supplies email and password. If the backend signals mfa_required: true, the component transitions to the MFA step, displaying a TOTP code field and an optional backup code toggle. On successful authentication, isAuthenticated is stored in localStorage and the user is redirected to the Dashboard.

RegisterView: Collects username, email, password, password_confirm, and optional domain. Client-side validation ensures both password fields match before submission.

DashboardView: Fetches and displays user profile data. Shows Outlook integration status with buttons to connect/disconnect and view the latest email. A modal collects the Azure Client ID and Client Secret required to initiate the OAuth flow.

EncryptionView: Provides both encrypt and decrypt panels in a single view. The algorithm selector dynamically hides or shows the relevant key/nonce fields. Hash algorithms hide the decrypt panel entirely since they are one-way functions.

CVEListView: Renders a paginated list of recent CVEs fetched from the backend. Each CVE card shows its ID, severity badge, CVSS score, description, published date, and a list of reference URLs.

AttackSurfaceView: Presents a domain input and a scan button. Results are rendered in an accordion layout with one panel per reconnaissance tool, showing the count of findings in the header for quick assessment.

MFASettingsView: Implements a multi-step MFA management wizard: idle → setup → verify → backup code display → disable. The QR code returned from the backend is rendered as a base64 PNG for scanning with an authenticator app.

ApiService: An Axios instance configured with baseURL http://localhost:8000/auth and withCredentials: true. Exports individual functions for each API call, keeping view components free from raw HTTP logic.

3. Class Interfaces

3.1 Server

3.1.1 Model

class CustomUser		
Represents a registered user of the Papillon platform. Persisted in the users table.		
Attributes		
`str`	`username`	`(primary key, max 150 chars, unique)`
`str`	`email`	`(unique, not null)`
`str`	`password`	`(PBKDF2+SHA256 hash)`
`str?`	`domain`	`(organisation domain for scoping scans)`
`datetime`	`created_at`	
`datetime`	`updated_at`	
`bool`	`is_active`	`(default True)`
`bool`	`mfa_enabled`	`(default False)`
`str?`	`mfa_secret`	`(Base32 TOTP seed, stored only after setup verification)`
`str?`	`mfa_backup_code`	`(hashed 6-digit single-use code)`
Methods		
`set_password(raw_password: str) -> None`		
// Hashes raw_password via Django make_password and stores result in self.password.		
`__str__() -> str`		
// Returns username.		
Edge Cases and Error Handling		
set_password() called with empty string → Django make_password stores an unusable hash; login will always return 401 for that account. Duplicate username on register → 400 {detail: 'Username already exists'}. Duplicate email on register → 400 {detail: 'Email already registered'}. mfa_secret written to DB before mfa_verify_setup succeeds → prevented by design; secret lives in session only until first successful OTP.		
Testability Notes		
Unit: set_password(raw) stores a value that check_password(raw, stored) returns True for. Unit: set_password('') produces an unusable hash (login must reject it).		

Integration: POST /register with duplicate username returns 400.

Integration: POST /register with duplicate email returns 400.

Unit: mfa_enabled defaults to False on a freshly created user.

Unit: mfa_secret is None until mfa_verify_setup completes successfully.

class EncryptedData

Records a single cryptographic operation performed by a user.

Attributes

int	id	(auto-increment primary key)
ForeignKey	user	(→ CustomUser, CASCADE)
str	algorithm	(AES-256-GCM | RSA-2048 | MD5 | SHA-1 | SHA-256 | SHA-512 | Base64)
str?	plaintext	(original text; stored for usability — see trade-offs)
str?	plaintext_preview	(first 100 chars)
str?	ciphertext	(base64-encoded output for AES/RSA/Base64)
str?	key	(base64 AES-256 key)
str?	nonce	(base64 12-byte GCM nonce)
str?	private_key	(PEM RSA private key)
str?	public_key	(PEM RSA public key)
str?	hash_result	(hex digest for MD5/SHA variants)
datetime	created_at	(auto-set on creation)

Methods

```
__str__() -> str
```

// Returns "<username> - <algorithm> - <created_at>".

Edge Cases / Error Handling

set_password() called with empty string → Django make_password stores an unusable hash; login will always return 401 for that account.

Duplicate username on register → 400 {detail: 'Username already exists'}.

Duplicate email on register → 400 {detail: 'Email already registered'}.

mfa_secret written to DB before mfa_verify_setup succeeds → prevented by design; secret lives in session only until first successful OTP.

Testability Notes

Unit: set_password(raw) stores a value that check_password(raw, stored) returns True for.

Unit: set_password('') produces an unusable hash (login must reject it).

Integration: POST /register with duplicate username returns 400.

Integration: POST /register with duplicate email returns 400.

Unit: mfa_enabled defaults to False on a freshly created user.

Unit: mfa_secret is None until mfa_verify_setup completes successfully.

class OutlookAccount

Stores a per-user Microsoft OAuth token set, encrypted at rest via Fernet.

Attributes

`OneToOneField`	`user`	`(→ CustomUser, CASCADE)`
`str?`	`_access_token`	`(Fernet-encrypted; accessed via access_token property)`
`str?`	`_refresh_token`	`(Fernet-encrypted; accessed via refresh_token property)`
`str?`	`_client_id`	`(Fernet-encrypted Azure app client ID)`
`str?`	`_client_secret`	`(Fernet-encrypted Azure app client secret)`
`str?`	`auth_tenant`	
`datetime?`	`expires_at`	
`bool`	`is_connected`	`(default False)`
`str?`	`outlook_email`	`(email address of the connected Microsoft account)`
`datetime`	`created_at`	
`datetime`	`updated_at`	

Methods

```
access_token (property getter) -> str?
```
// Decrypts and returns _access_token using Fernet cipher.

```
access_token (property setter, value: str) -> None
```
// Encrypts value and stores in _access_token.

```
refresh_token (property getter/setter) -> str?
```
// Same pattern as access_token.

```
client_id (property getter/setter) -> str?
```
// Same pattern as access_token.

```
client_secret (property getter/setter) -> str?
```
// Same pattern as access_token.

`_decrypt_value(value: str) -> str?`
`_encrypt_value(value: str) -> str?`
`__str__() -> str`
Edge Cases / Error Handling
access_token property getter called when _access_token is None → returns None without raising; get_latest_mail returns 400. Fernet key rotated in Vault after tokens stored → _decrypt_value raises InvalidToken; controller should return 400 and prompt re-authorisation. expires_at in the past → current implementation does not auto-refresh; Graph API call returns 401 which propagates as 400 to the frontend. OneToOneField constraint: second OutlookAccount for same user → update_or_create prevents duplicate; only one record per user exists.
Testability Notes
Unit: setting access_token = 'abc' then reading access_token returns 'abc' (round-trip through Fernet encrypt/decrypt). Unit: _decrypt_value(None) returns None without raising. Unit: _decrypt_value(corrupt_bytes) raises cryptography.fernet.InvalidToken. Integration: disconnect() deletes the OutlookAccount row; subsequent get_outlook_status returns is_connected=False. Integration: two save_client_id calls for the same user → update_or_create leaves exactly one OutlookAccount row.

3.1.2 Controller

class UserController
Handles user registration, login, logout, MFA setup, and MFA verification.
Attributes
`— (stateless view functions; Django session provides state) —`
Methods
`register(request: HttpRequest) -> JsonResponse`
// POST /auth/register/. Validates username uniqueness and email uniqueness,
// hashes password, creates CustomUser. Returns 201 on success.
`login(request: HttpRequest) -> JsonResponse`
// POST /auth/login/. Verifies email + password. If MFA enabled, stores
// mfa_pending_user and mfa_token in session and returns mfa_required: true.

```
    // Otherwise creates full session. Returns 200.
_generate_mfa_token(username: str) -> str
    // Generates a 32-char SHA-256-based token for the pending MFA session.
verify_mfa(request: HttpRequest) -> JsonResponse
    // POST /auth/mfa/verify/. Validates mfa_token against session, checks 5-min timeout.
    // Verifies TOTP code (or backup code). On success promotes session to full user session.
mfa_setup(request: HttpRequest) -> JsonResponse
    // POST /auth/mfa/setup/. Generates a new TOTP secret, stores it in session,
    // returns base64 PNG QR code and raw secret.
mfa_verify_setup(request: HttpRequest) -> JsonResponse
    // POST /auth/mfa/verify-setup/. Verifies first OTP against session secret.
    // On success writes secret to DB, sets mfa_enabled=True, generates backup code.
mfa_disable(request: HttpRequest) -> JsonResponse
    // POST /auth/mfa/disable/. Requires password confirmation. Clears mfa_secret,
    // mfa_enabled, and mfa_backup_code.
mfa_status(request: HttpRequest) -> JsonResponse
    // GET /auth/mfa/status/. Returns mfa_enabled flag for the current user.
logout(request: HttpRequest) -> JsonResponse
    // POST /auth/logout/. Flushes the entire session.
dashboard(request: HttpRequest) -> JsonResponse
    // GET /auth/dashboard/. Returns user profile data for the authenticated session.
```

Edge Cases / Error Handling

POST /login with missing email or password field → 400 'Email and password are required'.

POST /login for inactive user (is_active=False) → 401 (DoesNotExist branch).

POST /mfa/verify/ after 5-minute window → session.flush(), 401 'MFA suresi doldu'.

POST /mfa/verify/ with mismatched mfa_token → 401 'Gecersiz veya suresi dolmus MFA'.

POST /mfa/verify-setup/ without prior mfa_setup call (no session secret) → 400 'Once MFA setup baslatın'.

POST /mfa/disable/ with wrong password → 401 'Sifre yanlis'.

GET /dashboard/ without session → 401 'Not authenticated'.

Invalid JSON body on any POST → 400 'Invalid JSON'.

Testability Notes

Unit: _generate_mfa_token(username) returns a 32-character hex string.

Unit: two calls with same username at different times return different tokens (timestamp component).

Integration: full login → verify_mfa happy path creates session with user_id.

Integration: verify_mfa with expired session (mock time) returns 401.

Integration: verify_mfa with backup code rotates mfa_backup_code in DB and returns new_backup_code in response.

Integration: mfa_verify_setup with invalid OTP leaves mfa_enabled=False in DB.

Integration: logout flushes session; subsequent dashboard call returns 401.

class CryptoController

Handles text encryption, hashing, and decryption.

Attributes

```
  — (stateless view functions) —
```

Methods

```
  encrypt_text(request: HttpRequest) -> JsonResponse
    // POST /crypto/encrypt/. Requires session. Dispatches to the appropriate
    // cryptographic branch based on the algorithm field in the request body.
    // Creates an EncryptedData record and returns the output fields.
    //   AES-256-GCM: generates 256-bit key + 12-byte nonce; returns ciphertext, key, nonce.
    //   RSA-2048: generates keypair; returns ciphertext, private_key, public_key.
    //   MD5 / SHA-1 / SHA-256 / SHA-512: returns hex digest.
    //   Base64: returns base64-encoded string.
  decrypt_text(request: HttpRequest) -> JsonResponse
    // POST /crypto/decrypt/. Reverses AES-256-GCM (requires key + nonce),
    // RSA-2048 (requires private_key), or Base64. Hash algorithms return 400 error.
```

Edge Cases / Error Handling

algorithm field not in {AES-256-GCM, RSA-2048, MD5, SHA-1, SHA-256, SHA-512, Base64} → 400 'Unsupported algorithm'.

text field empty or missing → 400 'Text is required'.

AES decrypt: key or nonce missing → 400 'Key and nonce are required'.

AES decrypt: base64-decode of malformed key/nonce raises binascii.Error → 400 'AES Decryption failed'.

AES decrypt: authentication tag mismatch (wrong key) → cryptography.exceptions.InvalidTag → 400.

RSA decrypt: private_key missing → 400 'Private key is required'.

RSA decrypt: corrupt PEM → ValueError → 400 'RSA Decryption failed'.

Hash decrypt attempt → 400 'one-way hash, cannot decrypt'.

Request without valid session → 401 'Not authenticated'.

Testability Notes

Unit: encrypt then decrypt round-trip for AES-256-GCM returns original plaintext.

Unit: encrypt then decrypt round-trip for RSA-2048 returns original plaintext.

Unit: encrypt with Base64 then decrypt returns original string.

Unit: SHA-256 of known input matches expected hex digest.

Unit: decrypt with algorithm='MD5' returns 400 without DB access.

Unit: encrypt with algorithm='BLOWFISH' returns 400 without DB access.

Integration: AES decrypt with wrong key returns 400.

Integration: each encrypt call creates exactly one EncryptedData row.

class CVEController

Fetches and formats recent CVE records from the NIST NVD API.

Attributes

```
— (stateless view functions) —
```

Methods

```
latest_cves(request: HttpRequest) -> JsonResponse
    // GET /cve/latest/?limit=N. Queries NVD REST API v2.0 for CVEs published in
    // the last 60 days. Parses severity (CVSS v3.1 preferred, v2 fallback),
    // score, description, published date, and top 3 reference URLs.
    // limit is clamped to [1, 50]. Returns list of CVE objects.
```

Edge Cases / Error Handling

NVD API unreachable (timeout) → requests.RequestException caught; returns 500 {detail: 'Error fetching CVEs: ...'}.

NVD returns empty vulnerabilities list → returns 200 with cves=[].

limit query param missing → defaults to 10.

limit=0 → clamped to 1.

limit=999 → clamped to 50.

CVE entry has no cvssMetricV31 and no cvssMetricV2 → severity='UNKNOWN', score=0.0.

CVE entry has no English description → fallback 'No description available'.

Testability Notes

Unit: limit clamping — clamp(0,1,50)=1, clamp(999,1,50)=50, clamp(10,1,50)=10.

Unit: CVSS parsing prefers v3.1 over v2 when both present.

Unit: CVSS parsing falls back to v2 when v3.1 absent.

Unit: missing English description returns 'No description available'.

Integration (mock NVD): valid response returns list of CVE objects with id, severity, score, description, published, references.

Integration (mock NVD): RequestException returns 500.

class OutlookController

Manages Microsoft OAuth 2.0 flow and Outlook inbox access.

Attributes

```
CLIENT_ID: str    (loaded from Vault path papillon/outlook)
CLIENT_SECRET: str
TENANT_ID: str
REDIRECT_URI: str  (http://localhost:8000/outlook/callback)
```

Methods

```
save_client_id(request: HttpRequest) -> JsonResponse
  // POST /outlook/save-client-id/. Stores client_id and client_secret in session
  // temporarily for the OAuth flow.
authorize(request: HttpRequest) -> JsonResponse
  // GET /outlook/authorize/. Constructs Microsoft OAuth authorization URL
```

```
    // using the session-stored client_id and returns it to the frontend.
  callback(request: HttpRequest) -> HttpResponseRedirect
    // GET /outlook/callback/. Exchanges authorization code for access + refresh tokens.
    // Fetches the Microsoft Graph /me endpoint to get the Outlook email address.
    // Creates or updates OutlookAccount; redirects to frontend dashboard.
  disconnect(request: HttpRequest) -> JsonResponse
    // POST /outlook/disconnect/. Deletes the OutlookAccount record for the user.
  get_outlook_status(request: HttpRequest) -> JsonResponse
    // GET /outlook/status/. Returns is_connected flag and outlook_email.
  get_latest_mail(request: HttpRequest) -> JsonResponse
    // GET /outlook/latest-mail/. Calls Microsoft Graph messages endpoint,
    // returns subject, sender, received date, and body preview of the newest email.
```

Edge Cases / Error Handling

save_client_id with empty client_id or client_secret → 400.

authorize called without prior save_client_id (no session key) → 400 'Client ID not provided'.

callback with OAuth error param → 400 'Authorization failed: <error>'.

callback with no code param → 400 'No authorization code received'.

callback with expired/missing session (outlook_auth_user_id absent) → 401 'Session expired'.

Token exchange POST to Microsoft fails (network error) → 400 'Token exchange failed'.

Graph /me call fails → requests.RequestException → 400.

get_latest_mail with no connected account → 400 'Outlook account not connected'.

get_latest_mail with empty inbox → 200 {email: null, detail: 'No emails found'}.

disconnect with no existing OutlookAccount → 404.

Testability Notes

Unit: authorize builds URL containing correct scope and redirect_uri.

Unit: authorize without session client_id returns 400.

Integration (mock MS): full OAuth flow stores encrypted tokens and sets is_connected=True.

Integration: access_token stored in DB is not plaintext (Fernet ciphertext != original token).

Integration: get_latest_mail returns subject, from, received_date, preview fields.

Integration: disconnect removes OutlookAccount row; is_connected becomes False.

class AttackSurfaceController

Orchestrates domain reconnaissance by calling all analysis tool modules.

Attributes

```
— (stateless view functions) —
```

Methods

```
attack_surface_scan(request: HttpRequest) -> JsonResponse
  // POST /attack-surface/scan/. Resolves domain to IP via socket.gethostbyname.
  // Calls each tool module inside a try/except block so a single tool failure
  // does not abort the entire scan. Returns a results dict with keys:
  // domain, ip, dns_records, subdomains, whois, ssl_info, open_ports,
  // emails, admin_panels, robots_txt, ip_info.
```

Edge Cases / Error Handling

domain field empty or missing → 400 'Domain is required'.

socket.gethostbyname fails (nonexistent domain) → 400 'Invalid domain or unable to resolve'.

Individual tool module import fails (ImportError) → error string recorded in that result key; other modules continue.

Individual tool raises exception during execution → caught per-module; partial result returned, scan not aborted.

Domain resolves but all tools fail → 200 response with error strings in each key.

Request body is not valid JSON → 400 (json.JSONDecodeError bubble).

Testability Notes

Unit: empty domain returns 400 before DNS resolution.

Unit: unresolvable domain returns 400 'Invalid domain or unable to resolve'.

Integration (mock tools): one tool raising RuntimeError does not prevent other tools from running; its key contains an error string.

Integration (mock DNS): valid domain returns 200 with all expected keys: domain, ip, dns_records, subdomains, whois, ssl_info, open_ports, emails, admin_panels, robots_txt, ip_info.

Unit: results dict always contains all 11 keys regardless of tool failures.

class VaultService
Singleton service for reading and writing secrets in HashiCorp Vault.
Attributes
`_vault_client: hvac.Client?  (singleton; None if Vault unreachable)`
`_vault_initialized: bool`
`_secret_cache: Dict[str, dict]  (path → secret data)`
Methods
`get_secret(key: str, default: Any, vault_path: str?, mount_point: str) -> str?`
// Priority: Vault KV v2 → .env fallback → default.
// If VAULT_ENFORCE=True and Vault is unavailable, raises RuntimeError.
`get_secret_bool(key: str, default: bool, ...) -> bool`
`get_secret_int(key: str, default: int, ...) -> int`
`write_secret(path: str, data: dict, mount_point: str) -> bool`
// Writes data to Vault KV v2 at the given path.
`clear_cache() -> None`
// Clears _secret_cache (call after key rotation).
`health_check() -> dict`
// Returns vault_available, authenticated, initialized, sealed, enforce_mode.
`_get_vault_client() -> hvac.Client?`
// Creates Vault client using VAULT_TOKEN or AppRole (VAULT_ROLE_ID + VAULT_SECRET_ID).
`_read_vault_secret(path: str, mount_point: str) -> dict?`
// Reads KV v2 secret; caches result.
Edge Cases / Error Handling
Vault unreachable and VAULT_ENFORCE=True → get_secret raises RuntimeError. Vault unreachable and VAULT_ENFORCE=False → falls back to .env; if .env also missing, returns default value. get_secret_bool with string value 'true'/'1' → True; 'false'/'0' → False. get_secret_int with non-numeric Vault value → ValueError; returns default. write_secret to an uninitialized client → returns False. clear_cache called → subsequent get_secret re-reads from Vault (no stale cache).

Testability Notes

Unit: get_secret returns Vault value when Vault available (mock hvac).

Unit: get_secret falls back to os.environ when Vault unavailable.

Unit: get_secret returns default when both Vault and env absent.

Unit: get_secret raises RuntimeError when VAULT_ENFORCE=True and Vault down.

Unit: clear_cache empties _secret_cache dict.

Unit: health_check returns vault_available=False when client init fails.

3.1.3 AI Module Interfaces

class PasswordStrengthDetection
DjangoAPI that classifies password strength using a pre-trained ML model.
Attributes
`model: sklearn.Pipeline  (loaded from password_strength_model.pkl via joblib)`
Methods
`predict_password_strength(data: PasswordInput) -> dict`
// POST /predict. Extracts features from data.password via extract_features(),
// Runs model.predict(), returns strength_level (0/1/2) and strength_label.
`extract_features(password: str) -> dict`
// Computes length, uppercase count, lowercase count, digit count,
// Special char count, entropy, and pattern flags.

class MalwareDetection
DjangoAPI that classifies .exe files using a pre-trained ML model.
Attributes
`model: sklearn  (loaded from ember_model.pkl via joblib)`
Methods
`model.predict(features: bytes) -> dict`
// Receives raw file bytes, calls extract_features(bytez) to process data.
// Runs model.predict(), returns prediction (0/1) and probability (0,1)
`extract_features(bytez: bytes) -> dict`
// Processes raw byte data into a numerical feature vector compatible with the pipeline.
// Analyzes byte-level patterns, metadata, and structural characteristics of the uploaded file.
Edge Cases / Error Handling
Uploaded file is not a valid PE (.exe) → feature extractor raises an exception; returns 400 'Invalid PE file'. File size exceeds Django upload limit → 413 before reaching the view. model.predict returns probability exactly 0.5 → treated as benign (threshold < 0.5 for malware). model.pkl file missing at startup → joblib.load raises FileNotFoundError; service fails to start.
Testability Notes

Unit: extract_features on a known benign PE returns a feature vector of expected length.

Unit: model.predict on known benign sample returns label=0.

Unit: model.predict on known malicious sample returns label=1.

Unit: non-PE bytes input raises exception caught by view → 400 response.

Integration: POST /predict with valid PE file returns {label, probability}.

class IntrusionDetection

DjangoAPI that classifies network traffic using a pre-trained ML model.

Attributes

```
model: sklearn  (loaded from xgb_model.pkl via joblib)
encoder : sklearn (loaded from label_encoder.pkl via joblib)
```

Methods

```
model.predict(features: list) -> dict
```

// Receives a sequential list of extracted features (time-series or packet window).

Edge Cases / Error Handling

Feature list length does not match model's expected input dimensions → XGBoost raises ValueError; view returns 400.

Feature list contains non-numeric value → preprocessing raises TypeError → 400.

Unknown traffic category not in LabelEncoder classes → inverse_transform raises ValueError.

model.pkl or label_encoder.pkl missing at startup → FileNotFoundError; service fails.

Testability Notes

Unit: known 'Normal' feature vector returns label 'Normal'.

Unit: known attack feature vector returns correct attack category (e.g. 'DoS').

Unit: feature list with wrong length returns 400.

Unit: feature list with string value returns 400.

Integration: POST /predict with valid feature list returns {label, confidence}.

class PhishingDetector
Singleton ONNX Runtime inference class for phishing email classification.
Attributes
`_instance: PhishingDetector?  (class-level singleton reference)`
`_sess: ort.InferenceSession?  (ONNX Runtime session)`
Methods
`__new__(cls) -> PhishingDetector`
// Enforces singleton: loads ONNX model on first instantiation only.
`_load_model() -> None`
// Loads phishing_email_model.onnx via onnxruntime.InferenceSession.
`predict(email_text: str) -> dict`
// Runs inference; returns { is_phishing: bool, label: str, confidence: str }.
Edge Cases / Error Handling
Model file phishing_email_model.onnx missing → _load_model sets _sess=None; predict returns {error: 'Model not loaded'}. Singleton not yet instantiated when predict called → __new__ triggers _load_model automatically. email_text is empty string → ONNX inference still runs; returns SAFE or PHISHING. email_text contains non-UTF-8 bytes → numpy array dtype=object accepts it; result depends on model tokenisation. Concurrent requests → singleton pattern ensures only one InferenceSession exists; ONNX Runtime is thread-safe.
Testability Notes
Unit: PhishingDetector() called twice returns the same object instance (singleton). Unit: _sess=None (model missing) → predict returns dict with 'error' key. Unit: known phishing text returns {is_phishing: True, label: 'PHISHING'}.

Unit: known safe text returns {is_phishing: False, label: 'SAFE'}.

Unit: empty string input does not raise; returns valid dict.

Integration: POST /predict with phishing email body returns label='PHISHING'.

3.2 Client

3.2.1 View

class LoginView
Manages the two-step login flow including optional MFA verification.
Attributes
`formData: { email: string, password: string }`
`otpCode: string`
`mfaStep: boolean  (true when backend signals mfa_required)`
`mfaToken: string  (temporary token returned by backend)`
`useBackup: boolean (toggle for backup code path)`
`newBackupCode: string (displayed after backup code login)`
`errors: object`
`loading: boolean`
Methods
`handleChange(e: Event) -> void`
`handleSubmit(e: Event) -> void`
// Calls login(email, password). If mfa_required, sets mfaStep and mfaToken.
// On full success stores isAuthenticated in localStorage and navigates to /dashboard.
`handleMfaSubmit(e: Event) -> void`
// Calls verifyMfa(mfaToken, otpCode, useBackup). On success navigates to /dashboard.
Edge Cases / Error Handling
handleSubmit with empty email or password → validate() populates errors; no API call made. Backend returns mfa_required=true → mfaStep set to true; OTP form shown. handleMfaSubmit with empty otpCode → should display validation error before API call. verifyMfa returns new_backup_code (backup code path) → newBackupCode state set; one-time code shown to user. API call fails (network error) → error state set; loading=false. Already authenticated user navigates to /login → useEffect redirects to /dashboard immediately.
Testability Notes

Unit: handleSubmit with valid credentials sets loading=true during request.

Unit: mfa_required=true response sets mfaStep=true and stores mfaToken.

Unit: successful verifyMfa sets localStorage.isAuthenticated='true'.

Unit: failed login (401) populates errors state and does not navigate.

Unit: useBackup toggle switches OTP label to 'Backup Code'.

Integration: full login → MFA verify flow navigates to /dashboard.

class RegisterView

Handles new user registration with client-side validation.

Attributes

```
formData: { username: string, email: string, password: string,
            password_confirm: string, domain: string }
errors: object
loading: boolean
```

Methods

```
handleChange(e: Event) -> void
handleSubmit(e: Event) -> void
  // Validates passwords match, calls register(), navigates to /login on success.
validate() -> boolean
  // Returns false and populates errors if required fields are empty or passwords differ.
```

Edge Cases / Error Handling

Password and password_confirm do not match → validate() returns false; errors.password_confirm populated; no API call.

Required fields (username, email, password) empty → validate() blocks submission.

Backend returns 400 duplicate username → error displayed in form.

Backend returns 400 duplicate email → error displayed in form.

domain field left blank → sent as empty string; backend stores '' (optional field).

Testability Notes

Unit: validate() returns false when passwords differ.

Unit: validate() returns false when username is empty.

Unit: validate() returns true for all valid inputs.

Integration: successful register navigates to /login.

Integration: duplicate username response sets error state.

class DashboardView
Main authenticated page. Shows user profile and Outlook integration controls.
Attributes
`user: object?`
`outlookStatus: { is_connected: boolean, outlook_email: string }?`
`loadingOutlook: boolean`
`latestMail: { subject, from, from_name, received_date, preview }?`
`loadingMail: boolean`
`showClientIdModal: boolean`
`clientId: string`
`clientSecret: string`
`showHelpModal: boolean`
Methods
`useEffect (on mount) -> void`
// Checks localStorage for isAuthenticated; redirects to /login if absent.
// Calls getDashboard() and checkOutlookStatus() in parallel.
`handleLogout() -> void`
// Calls logout(), clears localStorage, navigates to /login.
`handleConnectOutlook() -> void`
// Saves client_id + client_secret then calls authorize(); redirects browser to auth_url.
`handleGetLatestMail() -> void`
// Calls get_latest_mail endpoint and sets latestMail state.
`handleDisconnectOutlook() -> void`
// Calls disconnect endpoint and refreshes Outlook status.
Edge Cases / Error Handling
localStorage missing isAuthenticated → useEffect redirects to /login immediately.
getDashboard() returns 401 (session expired) → clear localStorage, redirect /login.

handleConnectOutlook with empty clientId or clientSecret → no API call; show validation error.

handleGetLatestMail when is_connected=false → button disabled; no API call.

Outlook API returns {email: null} (empty inbox) → 'No emails found' message shown.

handleDisconnectOutlook fails (network error) → error toast; outlookStatus not changed optimistically.

Testability Notes

Unit: missing localStorage token redirects to /login without API call.

Unit: 401 from getDashboard clears localStorage and redirects.

Unit: successful getDashboard sets user state with username, email, domain.

Unit: is_connected=true shows Disconnect button and email display.

Unit: is_connected=false shows Connect button and hides email.

Integration: connect → authorize → callback → dashboard shows outlook_email.

class EncryptionView

Provides encrypt and decrypt panels for all supported algorithms.

Attributes

```
plaintext: string
algorithm: string  (default AES-256-GCM)
result: object?    (encrypt response)
loading: boolean
error: string?
decryptAlgorithm: string
ciphertext: string
decryptKey: string
decryptNonce: string
decryptPrivateKey: string
decryptResult: object?
decryptLoading: boolean
decryptError: string?
```

Methods

```
handleEncrypt() -> void
  // POSTs { text: plaintext, algorithm } to /crypto/encrypt/. Sets result.
handleDecrypt() -> void
```

// POSTs { ciphertext, algorithm, key?, nonce?, private_key? } to /crypto/decrypt/.
// Hash algorithms show an error without calling the backend.
`isHashAlgorithm(algorithm: string) -> boolean`
// Returns true for MD5, SHA-1, SHA-256, SHA-512.

Edge Cases / Error Handling

handleEncrypt with empty plaintext → 400 from backend; error state set.

handleDecrypt with hash algorithm selected → isHashAlgorithm() returns true; error shown without API call.

AES decrypt without key or nonce → 400 from backend.

RSA decrypt without private_key → 400 from backend.

Unknown algorithm value submitted → 400 'Unsupported algorithm' from backend.

Very large plaintext (>1 MB) → RSA-2048 OAEP limit (~214 bytes) → 400 'RSA Decryption failed' (encryption would also fail for oversized input).

Testability Notes

Unit: isHashAlgorithm('MD5') returns true; isHashAlgorithm('AES-256-GCM') returns false.

Unit: handleDecrypt with hash algorithm does not call ApiService.

Integration: AES-256-GCM encrypt then decrypt returns original plaintext.

Integration: RSA-2048 encrypt then decrypt returns original plaintext.

Integration: SHA-256 encrypt; decrypt attempt returns error without API call.

Integration: encrypt response for AES includes ciphertext, key, and nonce fields.

class CVEListView

Fetches and displays recent CVEs with severity filtering.

Attributes

`cves: Array<CVEObject>`
`loading: boolean`
`error: string?`
`limit: number  (default 10)`

Methods

`useEffect (on mount) -> void`
// Redirects to /login if not authenticated; calls fetchCVEs().
`fetchCVEs() -> void`
// GET /cve/latest/?limit=limit. Parses response and sets cves state.
`getSeverityClass(severity: string) -> string`
// Returns a CSS class name based on severity level for colour-coded badges.

Edge Cases / Error Handling

NVD API unavailable → backend returns 500; error message displayed in UI.

limit changed to 0 → backend clamps to 1; at least one CVE returned.

NVD returns empty list → 'No CVEs found' message shown.

CVE with UNKNOWN severity → badge rendered with neutral/grey styling.

User not authenticated → useEffect redirects to /login before fetch.

Testability Notes

Unit: getSeverityClass('CRITICAL') returns correct CSS class.

Unit: getSeverityClass('UNKNOWN') returns fallback CSS class.

Unit: missing localStorage token redirects to /login without API call.

Integration (mock API): fetchCVEs populates cves state array.

Integration (mock API): changing limit triggers new fetchCVEs call with updated param.

class AttackSurfaceView
Collects a domain name and displays multi-category scan results in an accordion layout.
Attributes
`domain: string`
`results: object?  (dns_records, subdomains, whois, ssl_info, open_ports,`
`emails, admin_panels, robots_txt, ip_info)`
`loading: boolean`
`error: string?`
`pdfLoading: boolean`
Methods

`handleScan() -> void`
// POST /attack-surface/scan/ with { domain }. Sets results on success.
`handleDownloadPDF() -> void`
// Uses jsPDF + jsPDF-autotable to generate a PDF report of the scan results.
`AccordionSection({ title, count, children, defaultOpen }) -> JSX`
// Internal component; renders a collapsible panel with a badge showing count.

Edge Cases / Error Handling

handleScan with empty domain field → 400 from backend; error shown.

handleScan with invalid/unresolvable domain → 400 'Invalid domain'; error shown.

Scan returns partial results (some tools errored) → accordion sections with error messages rendered; PDF export still works.

handleDownloadPDF before scan results exist → button disabled or no-op.

Very long scan (subdomain enumeration) → loading spinner shown; UI remains responsive during wait.

Testability Notes

Unit: handleScan with empty domain does not call ApiService.

Unit: loading=true set immediately on handleScan; false after response.

Integration: valid domain scan returns results object with all 11 keys.

Integration: handleDownloadPDF triggers browser download with filename scan_report.pdf.

Unit: AccordionSection renders badge count equal to array length of each result key.

class MFASettingsView

Multi-step wizard for enabling and disabling TOTP-based MFA.

Attributes

`mfaEnabled: boolean`
`loading: boolean`
`setupData: { qr_code: string, secret: string }?`
`otpCode: string`
`disablePassword: string`
`backupCode: string  (shown once after successful setup)`
`message: { type: string, text: string }`

`step: string  (idle | setup | verify | backup | disable)`
Methods
`fetchMfaStatus() -> void`
// GET /auth/mfa/status/. Sets mfaEnabled and step.
`handleStartSetup() -> void`
// POST /auth/mfa/setup/. Stores returned qr_code and secret in setupData.
// Transitions step to "verify".
`handleVerifySetup() -> void`
// POST /auth/mfa/verify-setup/ with otpCode. On success transitions to "backup"
// and stores the one-time backup code.
`handleDisableMfa() -> void`
// POST /auth/mfa/disable/ with disablePassword. Resets state to idle.
Edge Cases / Error Handling
handleStartSetup called when mfa_enabled=true → should be unreachable (button hidden); backend returns error if called directly. handleVerifySetup with empty otpCode → should show validation error; no API call. OTP verified correctly → backup code shown exactly once; navigating away loses it (no persistence). handleDisableMfa with wrong password → 401; error shown; step remains 'disable'. mfa_setup session secret expires (server restart) → verify-setup returns 400; user must restart setup flow. fetchMfaStatus called without session → 401; redirect to /login.
Testability Notes
Unit: fetchMfaStatus 401 clears localStorage and redirects to /login. Unit: successful mfa_setup response sets setupData.qr_code and setupData.secret. Unit: successful mfa_verify_setup sets step='backup' and backupCode state. Unit: failed verify (400) keeps step='verify' and shows error message. Integration: full setup flow → idle → setup → verify → backup transitions. Integration: disable with correct password resets step to 'idle', mfaEnabled=false.

class ApiService
Axios instance and exported functions for all backend API calls.
Attributes
`API: AxiosInstance (baseURL: http://localhost:8000/auth, withCredentials: true)`
Methods
`register(username, email, password, domain?) -> AxiosPromise`
`login(email, password) -> AxiosPromise`
`logout() -> AxiosPromise`
`getDashboard() -> AxiosPromise`
`mfaSetup() -> AxiosPromise`
`mfaVerifySetup(otp_code) -> AxiosPromise`
`mfaDisable(password) -> AxiosPromise`
`verifyMfa(mfa_token, otp_code, use_backup?) -> AxiosPromise`
`mfaStatus() -> AxiosPromise`
Edge Cases / Error Handling.
withCredentials=true required for session cookie; missing it causes all authenticated endpoints to return 401. CORS preflight rejected (backend CORS_ALLOWED_ORIGINS not set) → network error before any response. verifyMfa called with use_backup=undefined → backend receives false (default); TOTP path taken. Network offline → AxiosError with no response; caller must handle undefined response. baseURL hardcoded to http://localhost:8000/auth → production deployments require environment variable substitution
Testability Notes
Unit: all methods call the correct HTTP verb and path (e.g. login → POST /auth/login/, getDashboard → GET /auth/dashboard/). Unit: verifyMfa passes use_backup boolean in request body. Unit: Axios instance has withCredentials=true configured. Integration: login response sets session cookie; subsequent getDashboard succeeds. Integration: logout invalidates session; getDashboard returns 401.

4. System Workflows

This section describes the dynamic behaviour of the system's core processes and provides the algorithmic logic (pseudocode) for key operations. To fully understand the system's design, these workflows should be reviewed in conjunction with the visual models and supplementary materials provided at the end of this document. Specifically:

- For the static structure, object-oriented design, and relationships of the components mentioned below, please refer to the **Class Diagrams in Appendix A**.
- For the step-by-step visual representation of the message passing and interactions, refer to the **Sequence Diagrams in Appendix B**.
- For database schemas, configuration details, and additional technical specifications, please see the **subsequent appendices**.

4.1 Authentication and MFA Workflow

The authentication process follows a two-step validation flow if Multi-Factor Authentication (MFA) is enabled.

(For the detailed sequence of operations, see Figure B.1 in Appendix B. To view the related class structures, refer to Figure A.3: Model Layer Class Diagram and Figure A.4: Controller Layer Class Diagram in Appendix A.)

```
FUNCTION login(email, password):
    user = Database.find_user(email)
    IF NOT user OR NOT verify_hash(password, user.password):
        RETURN Error("Invalid credentials")

    IF user.mfa_enabled == TRUE:
        mfa_token = generate_temp_token()
        Session.set("mfa_pending", mfa_token)
        RETURN RequireMFA(mfa_token)
    ELSE:
        Session.create(user.id)
        RETURN Success("Logged in")
```

4.2 Cryptography Workflow

Users can encrypt or decrypt data via the CryptoController. The controller dynamically routes the request to the appropriate cryptographic library function based on the selected algorithm and records the operation.

(For the sequence diagram, see Figure B.2 in Appendix B. For the underlying class structures, refer to Figure A.4: Controller Layer Class Diagram.)

```
FUNCTION encrypt_text(text, algorithm):
    IF algorithm == "AES-256-GCM":
        key, nonce = generate_aes_parameters()
```

```
        ciphertext = AES_GCM_Encrypt(text, key, nonce)
        Database.save(EncryptedData(ciphertext, key, nonce,
algorithm))
        RETURN ciphertext, key, nonce
```

4.3 Attack Surface Reconnaissance Workflow

The attack surface scanner takes a domain, resolves it, and orchestrates a series of reconnaissance tools. To ensure responsiveness, missing optional dependencies in individual tools do not halt the entire scan.

(For the sequence diagram, see Figure B.3 in Appendix B. To see how the controller interacts with these tools, refer to Figure A.7: AttackSurfaceController → Tools Class Diagram.)

```
FUNCTION attack_surface_scan(domain):
    ip_address = resolve_dns(domain)
    results = {}

    TRY results['dns'] = get_dns_records(domain) CATCH err
CONTINUE
    TRY results['ports'] = scan_ports(ip_address) CATCH err
CONTINUE
    TRY results['ssl'] = scan_ssl_cert(domain) CATCH err CONTINUE
    // Executes remaining tools (WHOIS, Subdomains, Emails, etc.)

    RETURN JSON(results)
```

4.4 CVE Fetching Workflow

The platform proactively queries the NIST NVD API to fetch the latest vulnerabilities, parsing the CVSS scores to categorize severity.

(For the sequence diagram, see Figure B.4 in Appendix B. For the class structure, refer to Figure A.4: Controller Layer Class Diagram.)

```
FUNCTION fetch_latest_cves(limit):
    date_range = current_date - 60_days
```

```
    response =
HTTP.GET("https://services.nvd.nist.gov/rest/json/cves/2.0",
params={date: date_range})


    cve_list = []
    FOR item IN response.cves:
        score = extract_cvss_v3(item) OR extract_cvss_v2(item)
        cve_list.append(format_cve(item.id, score,
item.description))


    RETURN cve_list[0:limit]
```

4.5 Outlook Integration Workflow

To analyse emails for phishing, users link their Microsoft accounts using an OAuth 2.0 authorization code flow. Sensitive tokens are encrypted at rest.

(For the sequence diagram, see Figure B.5 in Appendix B. For the related object representations, see Figure A.3: Model Layer Class Diagram and Figure A.4: Controller Layer Class Diagram.)

```
FUNCTION outlook_callback(auth_code):
    tokens = MicrosoftOAuth.exchange_code(auth_code)
    user_email = MicrosoftGraph.get_me(tokens.access_token)

    encrypted_access = Fernet.encrypt(tokens.access_token)
    encrypted_refresh = Fernet.encrypt(tokens.refresh_token)

    Database.save_or_update(OutlookAccount(encrypted_access,
encrypted_refresh, user_email))
    RETURN Redirect("/dashboard")
```

4.6 Phishing Detection Workflow

The phishing detection module utilizes a pre-trained model exported in ONNX format. To avoid the overhead of loading the model into memory on every request, the PhishingDetector class is implemented using the Singleton pattern.
(For the static structure and relationships of all AI components, refer to Figure A.1: Class Diagram for AI Modules in Appendix A. For the step-by-step dynamic execution, see Figure B.6: Sequence Diagram for AI-Driven Phishing Detection Inference in Appendix B.)

```
FUNCTION predict_phishing(email_text):
    IF PhishingDetector.model IS NULL:
        PhishingDetector.model = ONNXRuntime.load("model.onnx")

    // Run inference session
    prediction, confidence =
PhishingDetector.model.run(email_text)

    RETURN format_result(prediction, confidence)
```

4.7 Password Strength Evaluation Workflow

The password strength predictor handles requests asynchronously via a dedicated FastAPI endpoint. Upon receiving a raw password, it first routes the input through a feature extractor to calculate metrics before feeding the structured feature set into the serialized machine learning model.

(For the object-oriented design of this module, refer to Figure A.1: Class Diagram for AI Modules in Appendix A. For the asynchronous execution flow, see Figure B.7: Sequence Diagram for Machine Learning-Based Password Strength Evaluation in Appendix B.)

```
FUNCTION predict_password_strength(password):
    // Step 1: Extract numerical features from raw text
    features = extract_features(password)

    // Step 2: Run inference using the pre-loaded joblib ML model
    strength_level = ML_Model.predict(features)

    // Step 3: Map the output to a human-readable label
    strength_label = map_to_label(strength_level) // 0: Weak, 1:
Normal, 2: Strong

    RETURN JSON({ strength_level: strength_level, strength_label:
strength_label })
```

5. Appendix

5.1 Appendix A: Class Diagrams

This appendix provides the detailed UML class diagrams that illustrate the static structure, relationships, and responsibilities of the Papillon system components.

A.1 AI Modules Class Diagram

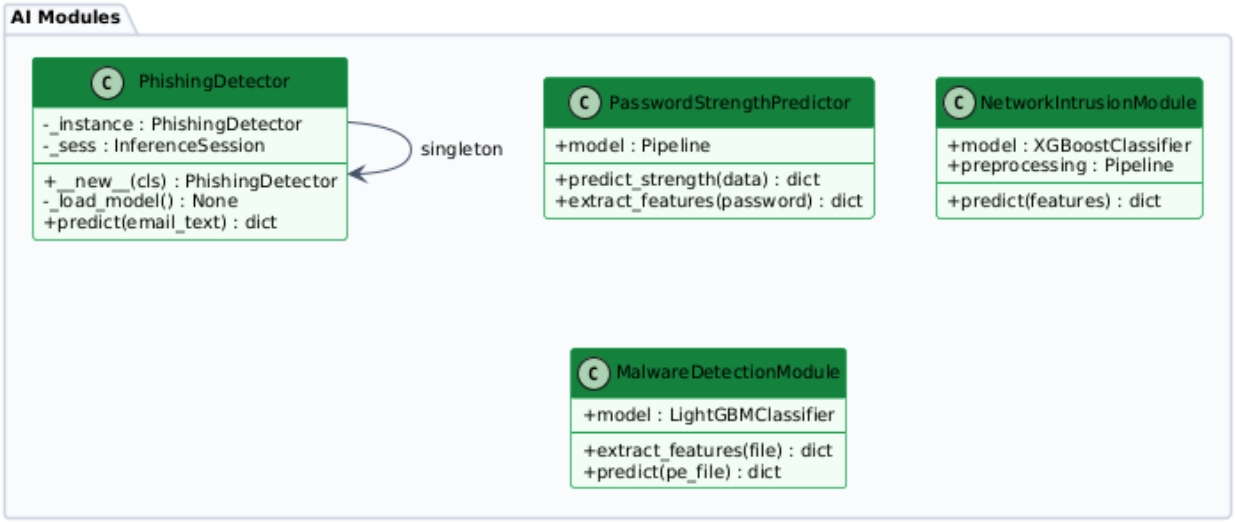


Figure A.1: Class Diagram for AI Modules

A.2 Frontend Class Diagram

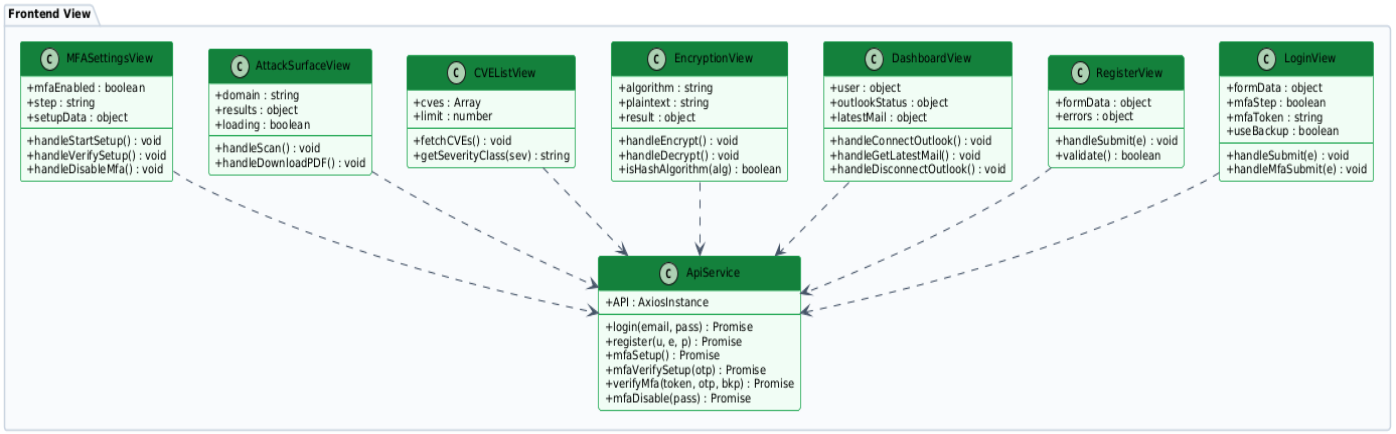


Figure A.2: Class Diagram for Frontend Views

A.3 Model Layer Class Diagram

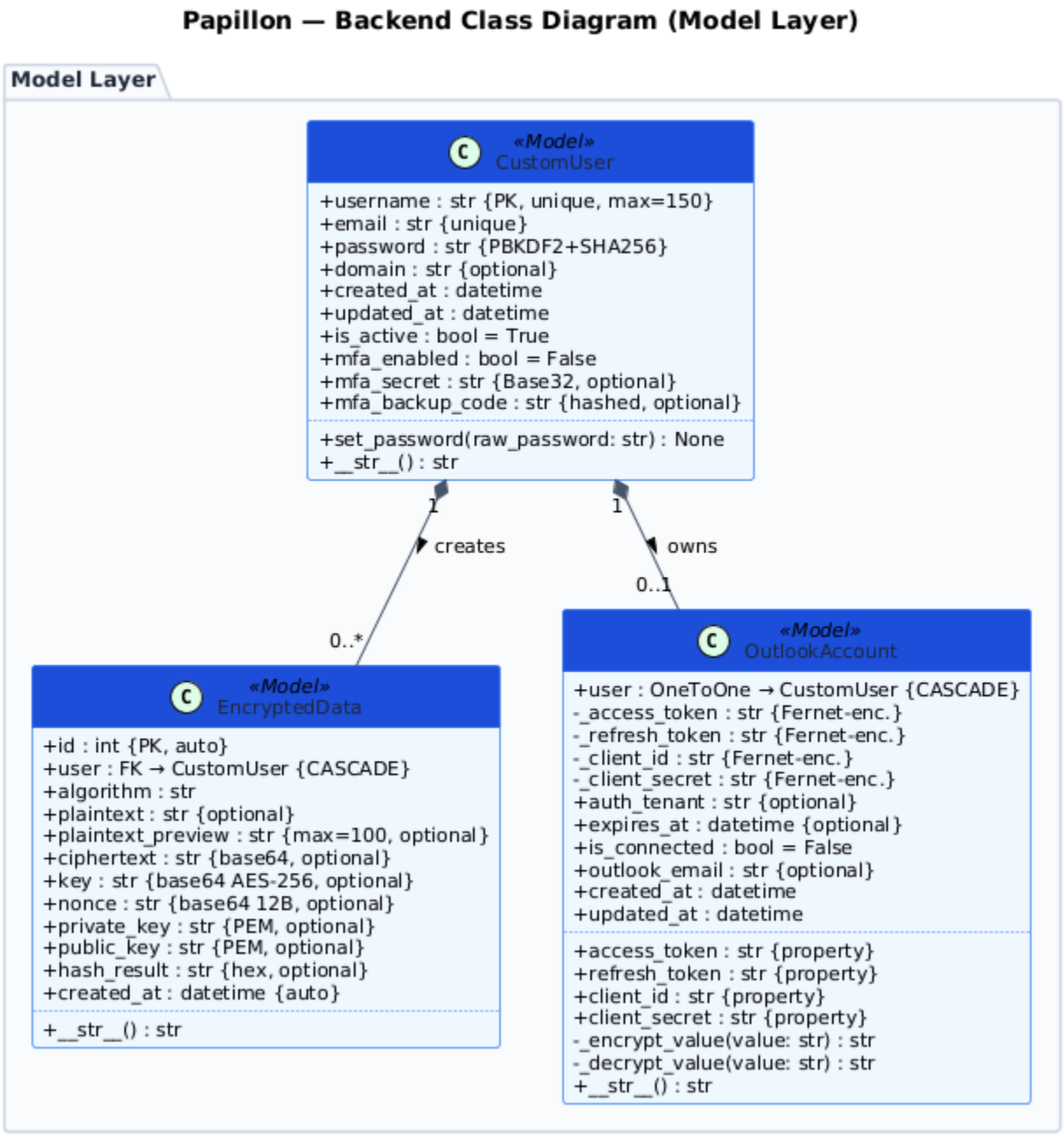


Figure A.3: Class Diagram for Model Layer

A.4 Controller Layer Class Diagram

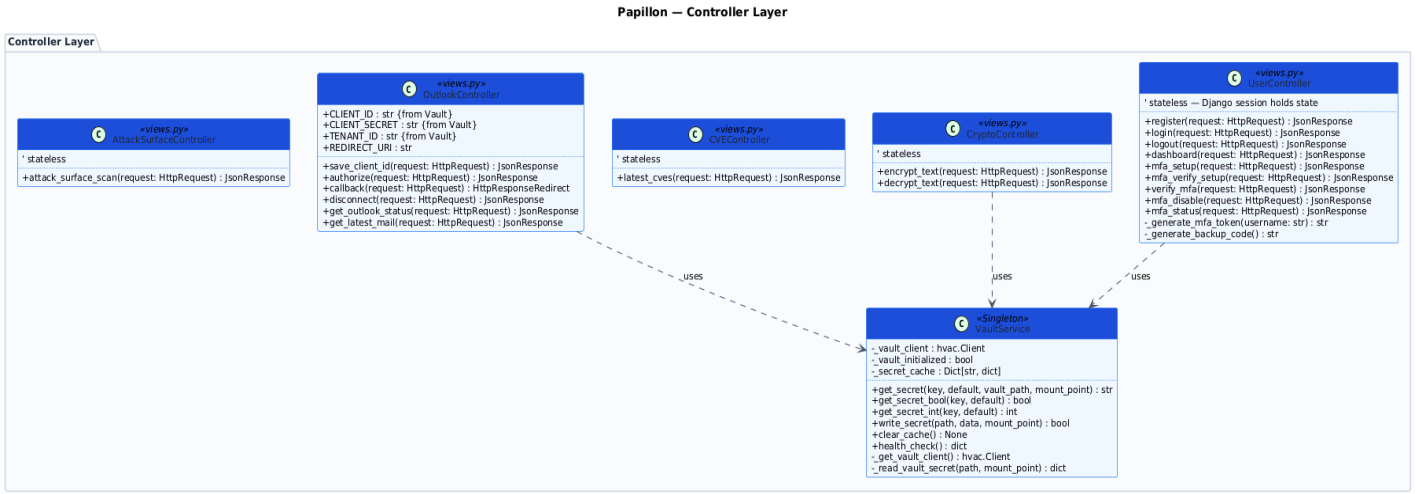


Figure A.4: Class Diagram for Controller Layer

A.5 DNS / Network Tools Class Diagram

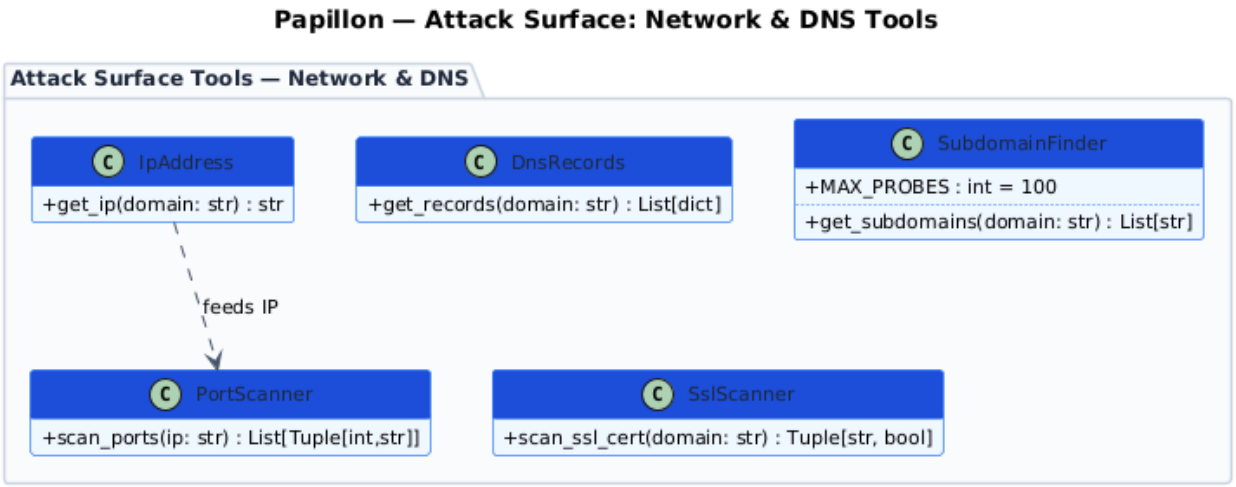


Figure A.5: Class Diagram for DNS and Network Analysis Tools

A.6 Web Reconnaissance Tools Class Diagram

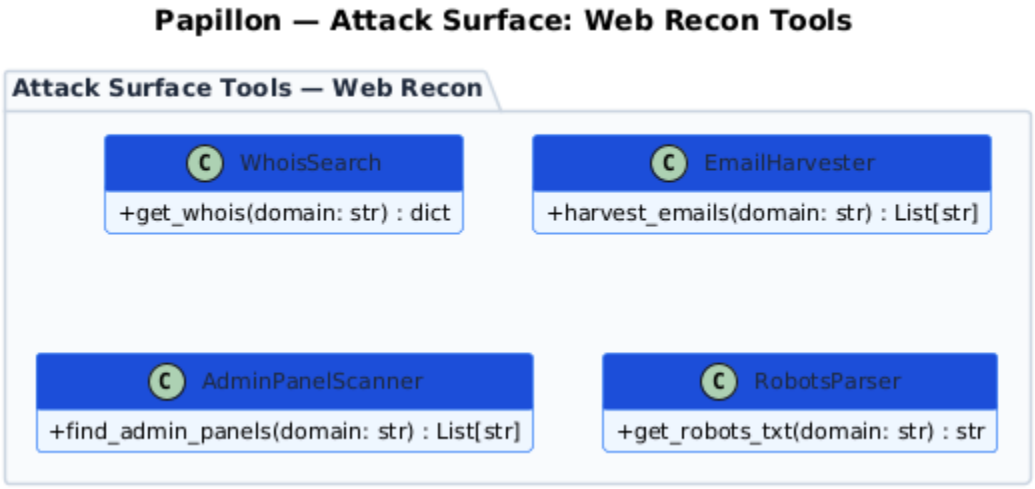


Figure A.6: Class Diagram for Web Reconnaissance Tools

A.7 AttackSurfaceController → Tools Class Diagram

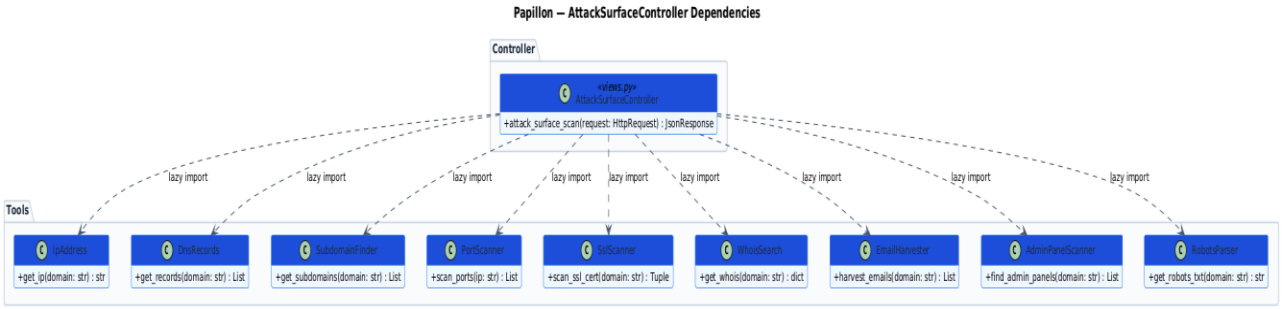


Figure A.7: Class Diagram for AttackSurfaceController and Tool Dependencies

5.2 Appendix B: Sequence Diagrams

This appendix provides the detailed UML sequence diagrams that illustrate the dynamic behavior, workflow, and message passing between the Papillon system components.

B.1 Authentication and MFA Workflow

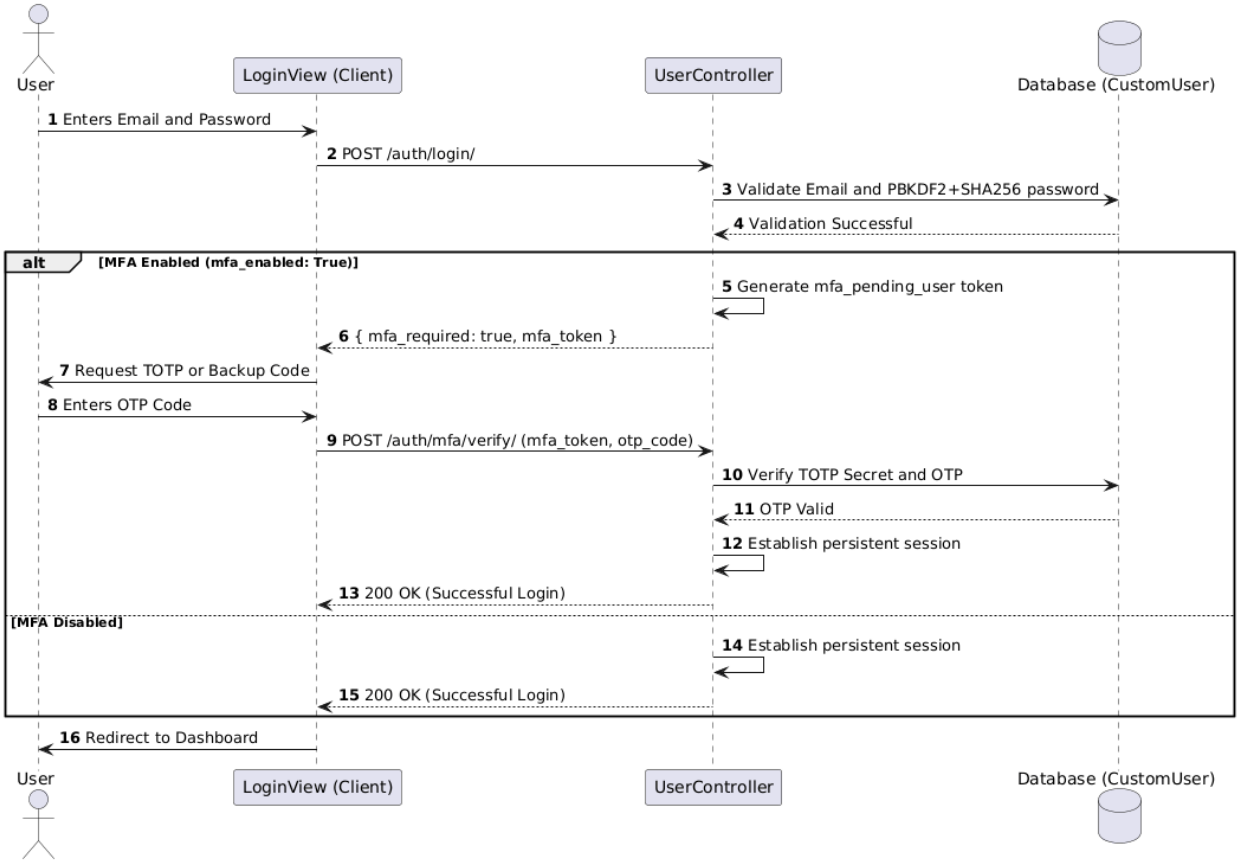


Figure B.1: Sequence Diagram for Authentication and MFA Workflow

B.2 Cryptography Workflow

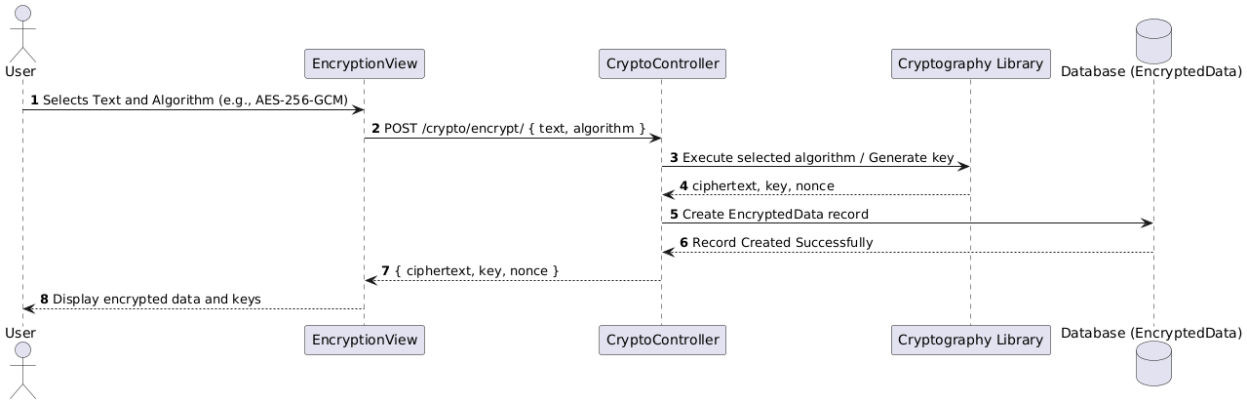


Figure B.2: Sequence Diagram for Cryptographic Operations (Encryption and Decryption).

B.3 Attack Surface Reconnaissance Workflow

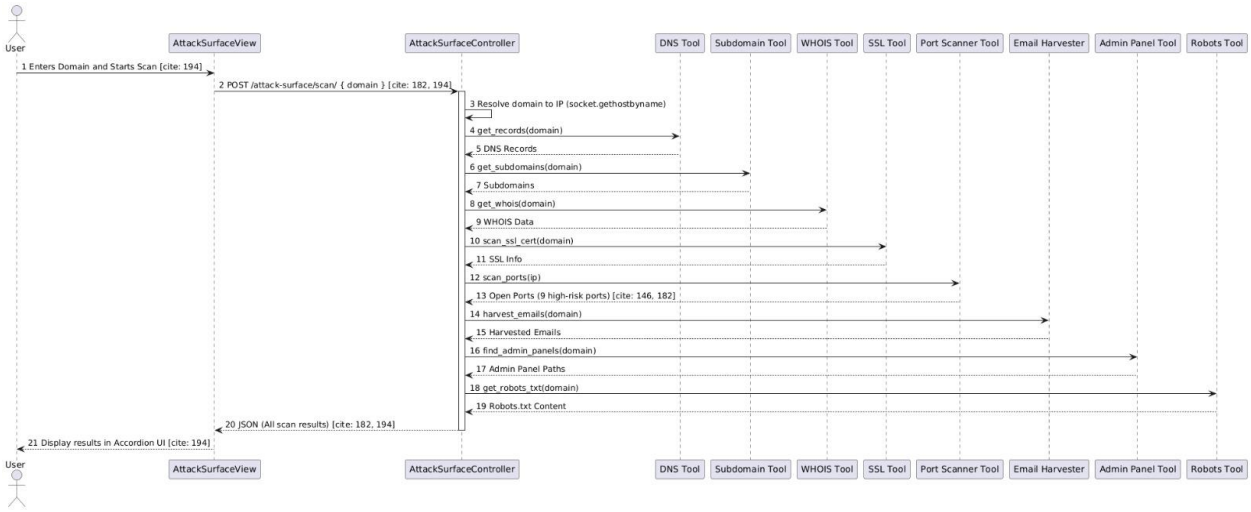


Figure B.3: Sequence Diagram for Attack Surface Analysis and Tool Orchestration.

B.4 CVE Fetching Workflow

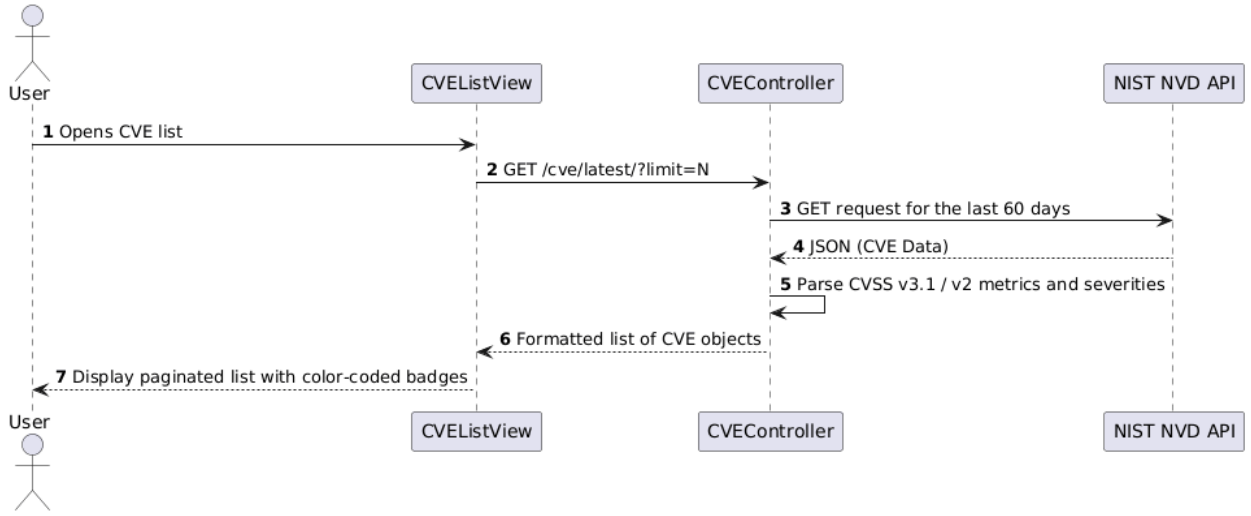


Figure B.4: Sequence Diagram for Fetching and Processing Latest CVE Records.

B.5 Outlook Integration Workflow

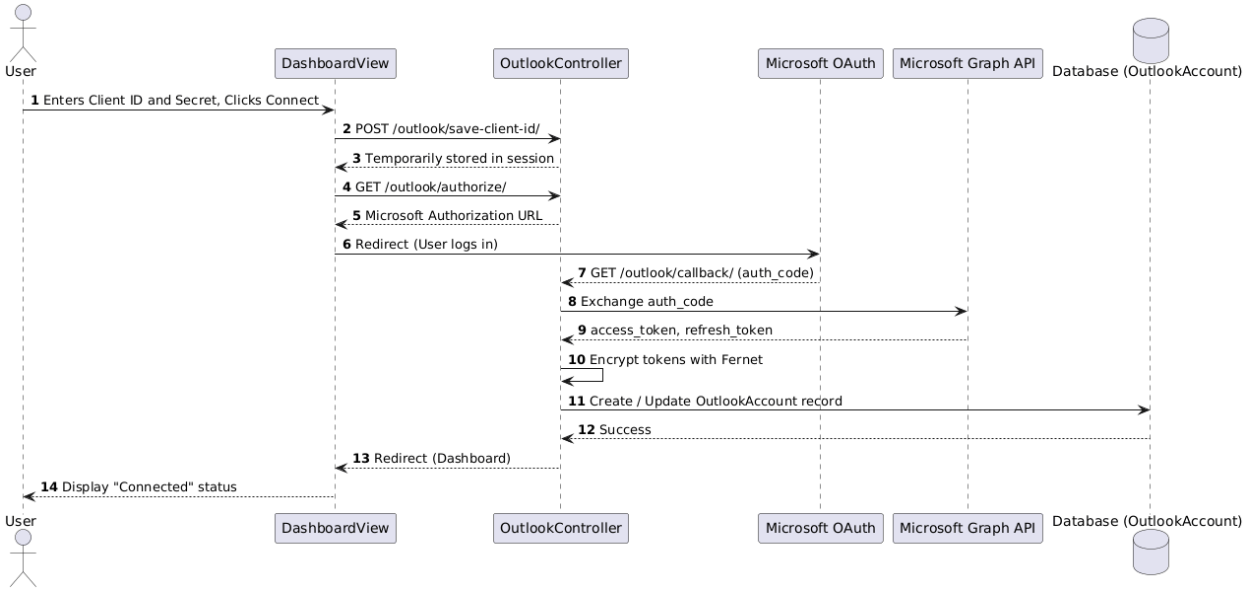


Figure B.5: Sequence Diagram for Microsoft Outlook OAuth 2.0 Integration.

B.6 Phishing Detection Workflow

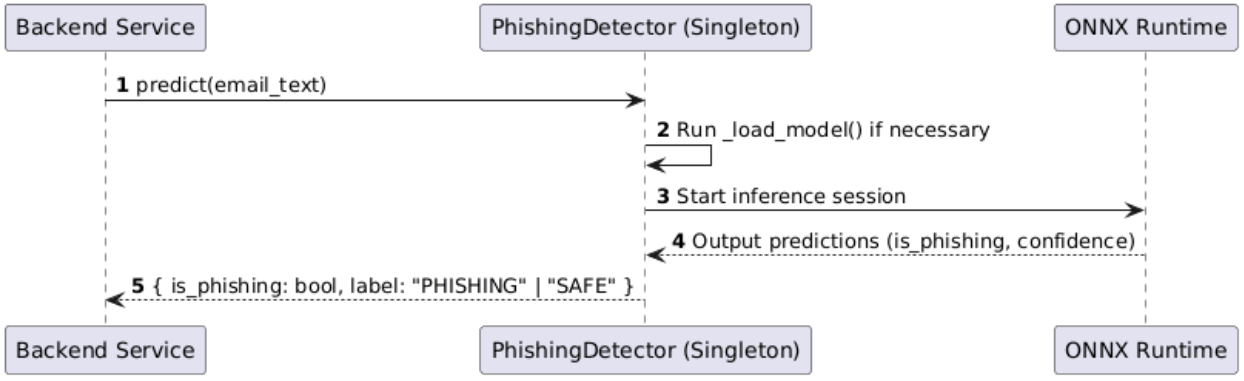


Figure B.6: Sequence Diagram for AI-Driven Phishing Detection Inference.

B.7 Password Strength Workflow

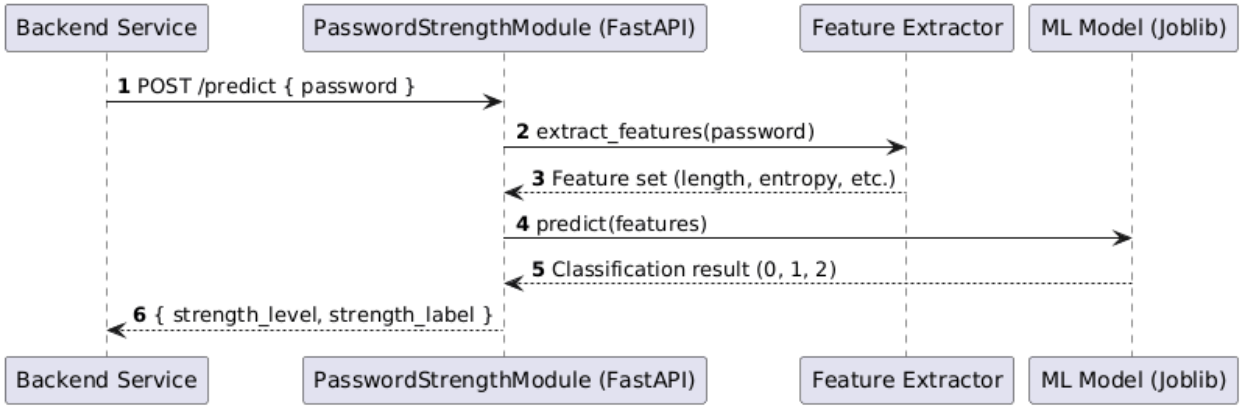


Figure B.7: Sequence Diagram for Machine Learning-Based Password Strength Evaluation.

B.8 Intrusion Detection Workflow

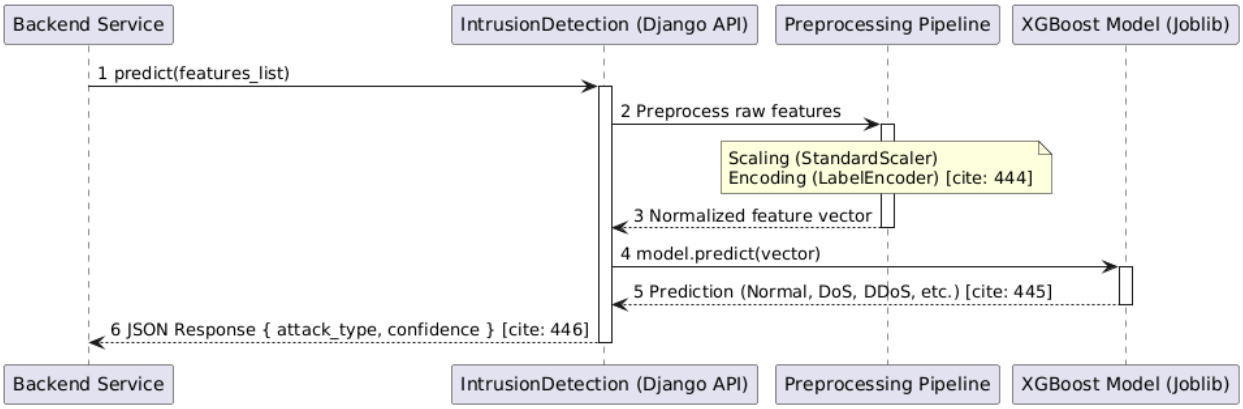


Figure B.8: Sequence Diagram for Intrusion Detection Workflow.

B.9 Password Malware Detection Workflow

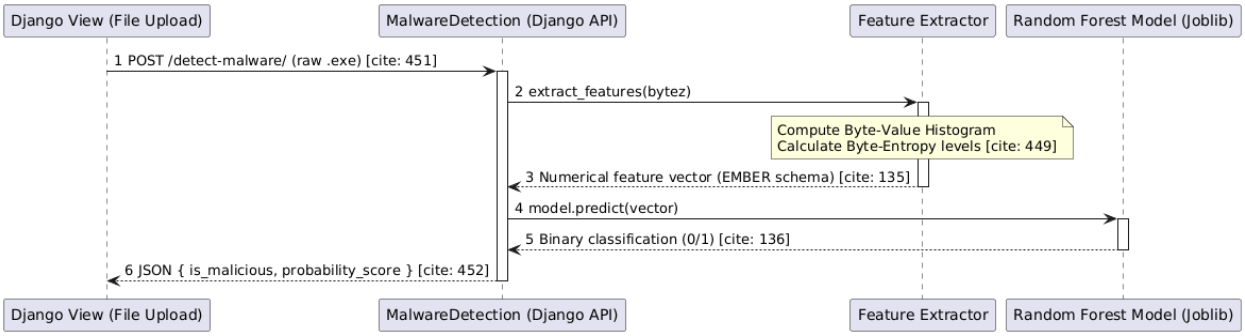


Figure B.9: Sequence Diagram for Malware Detection Workflow.

5.3 Appendix C: UML Activity Diagrams

This appendix presents the UML activity diagrams that illustrate the workflows and process flows of the Papillon system. The diagrams show the sequence of actions and decision points involved in the execution of the system's core functionalities.

C.1 Multi-Factor Authentication Setup and Management Activity Diagram

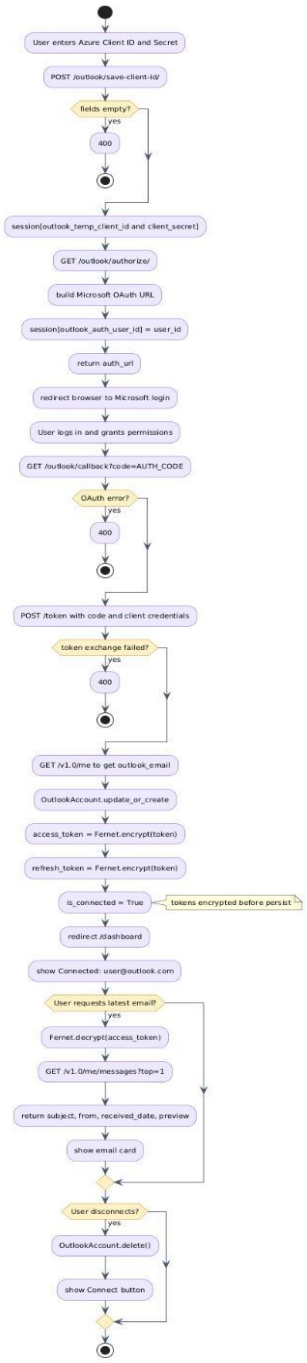


Figure C.1: Multi-Factor Authentication Setup and Management Activity Diagram.

C.2 Attack Surface Scanning Activity Diagram

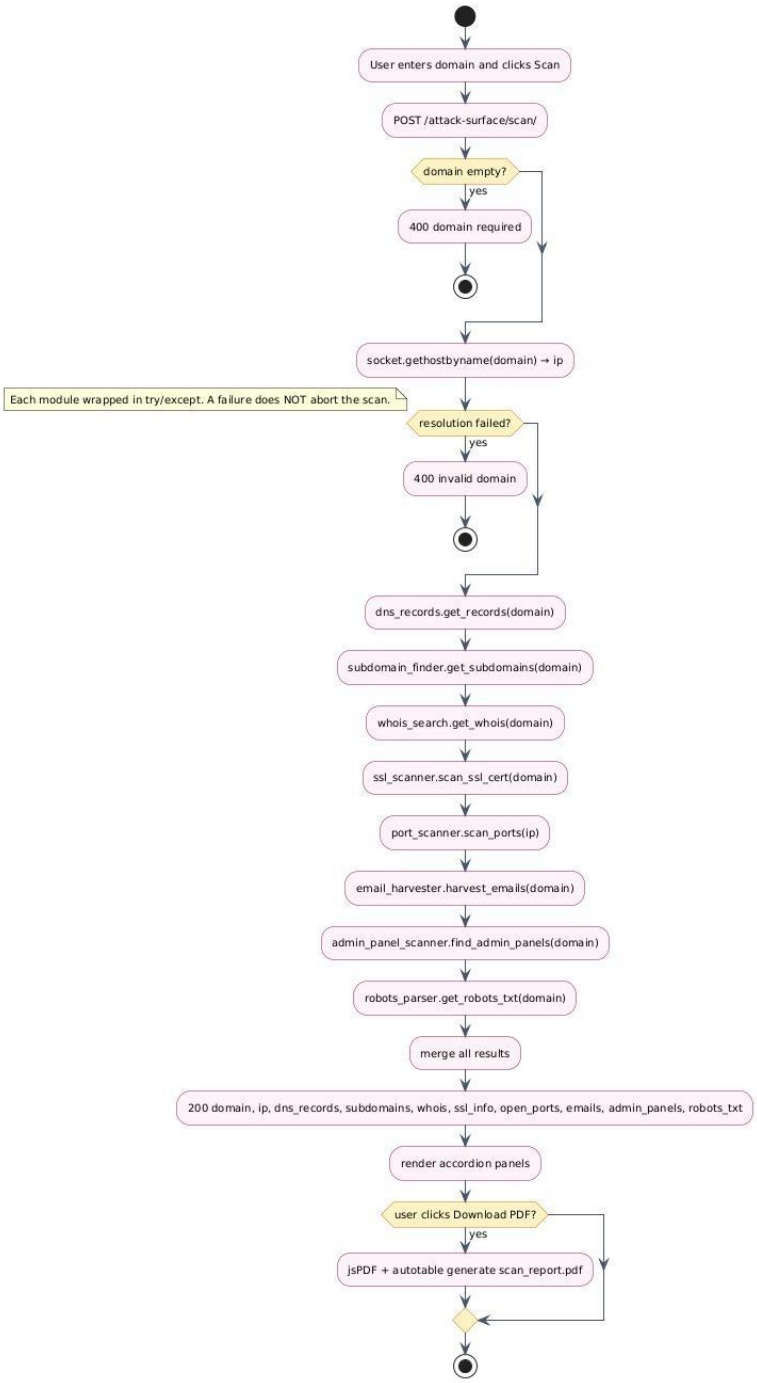


Figure C.2: Attack Surface Scanning Activity Diagram

C.3 Microsoft Outlook OAuth Integration Activity Diagram

Figure C.3: Microsoft Outlook OAuth Integration Activity Diagram

C.4 Authentication & MFA Activity Diagram

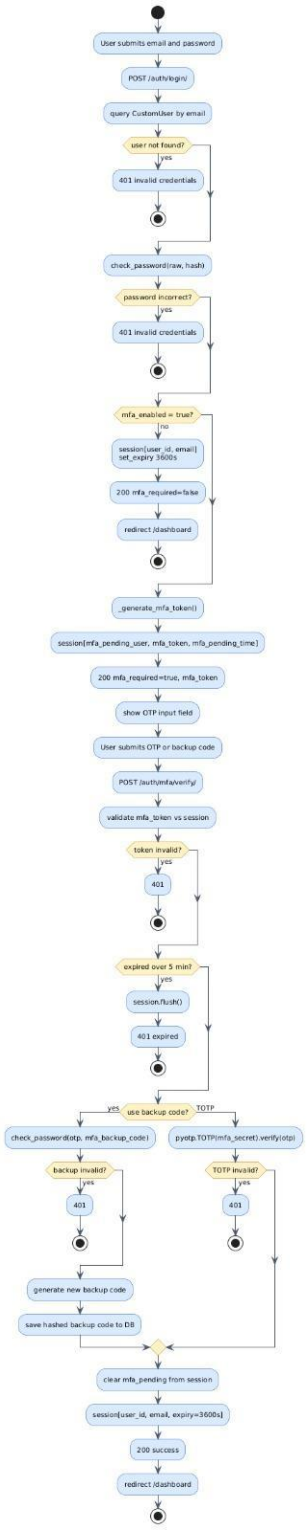


Figure C.4: Authentication & MFA Activity Diagram

C.5 Encryption Service Diagram

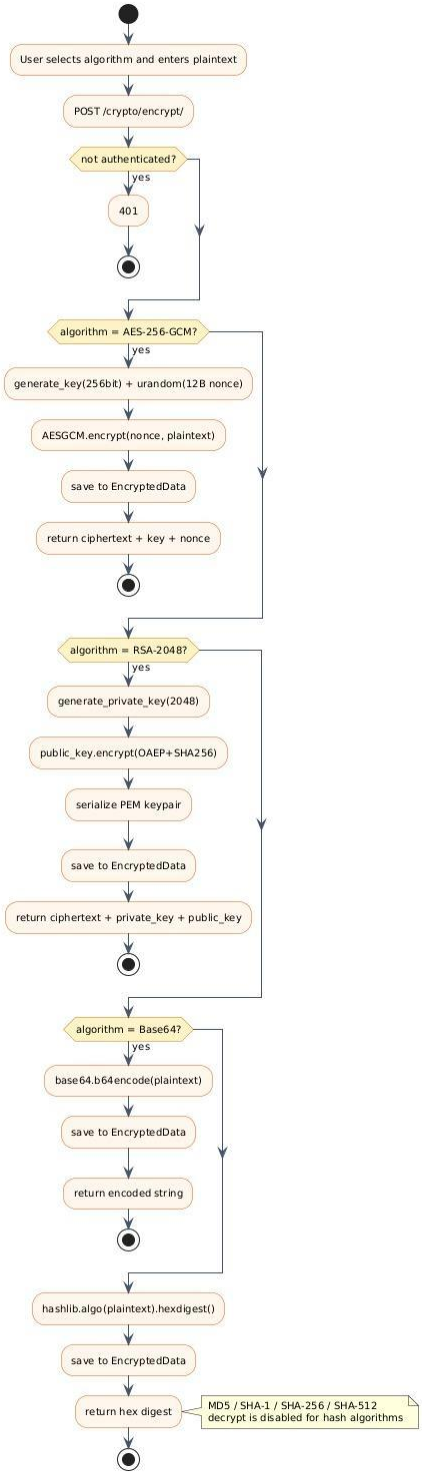


Figure C.5: Encryption Service Diagram

5.4 Appendix D: Database Schema and Entity-Relationship Models

This appendix provides the physical database schema and Entity-Relationship (ER) models for the Papillon system. The relational database is managed via MySQL, with all schema migrations and object-relational mapping (ORM) handled strictly by the Django framework.

The diagram below outlines the foundational tables of the system—focusing on user management, cryptographic operation logs, and external OAuth integrations—along with their primary keys, data types, and relational constraints (e.g., Foreign Key and One-to-One linkages).

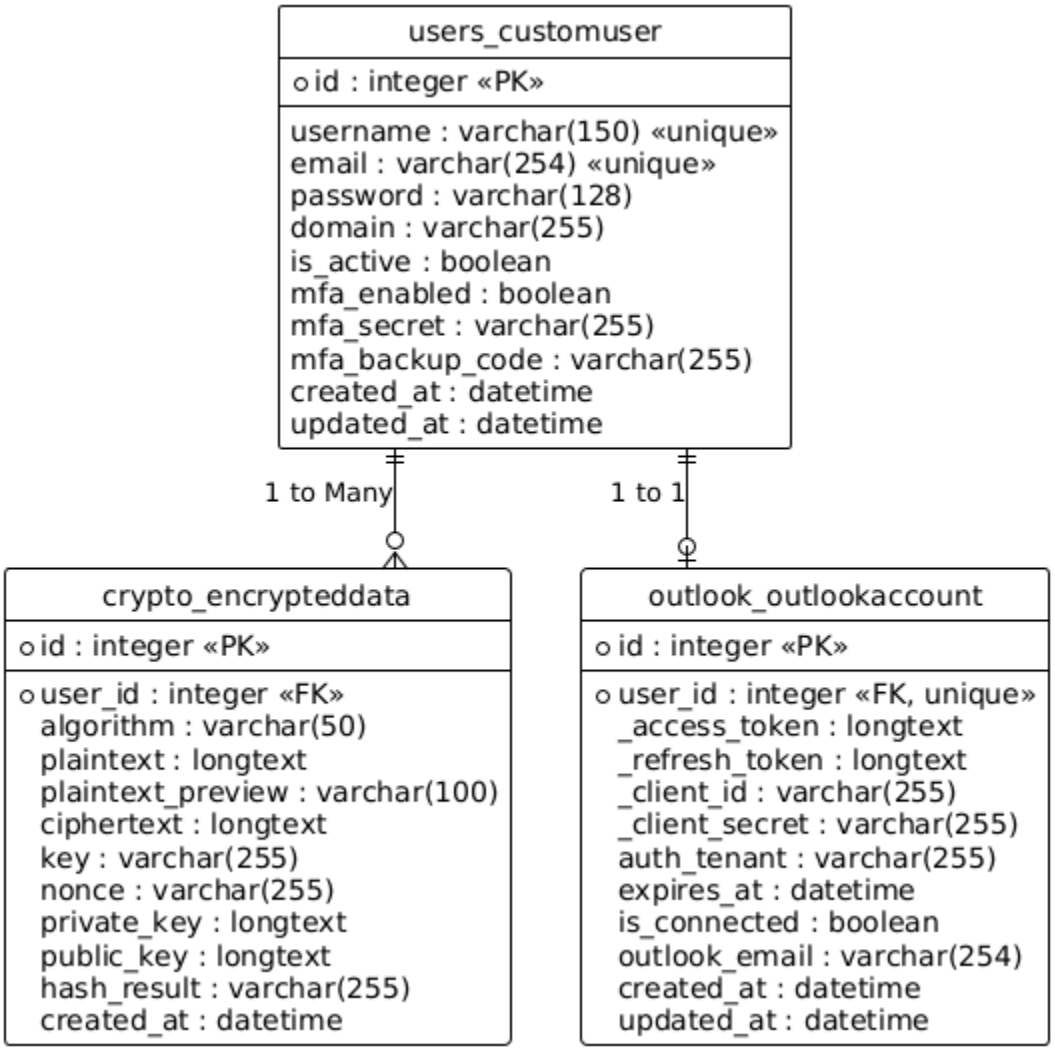


Figure D: Database Schema and Entity-Relationship Model

5.5 Appendix E: Environment Configuration / Base Settings

This appendix outlines the base environment configuration variables (.env) required to initialize and run the Papillon backend services in a local development environment. It defines the database connection parameters, Django core settings, and external API integrations.

As specified in the architectural trade-offs (Section 1.1.4) and the *`VaultService`* class interface, this configuration file acts as the primary configuration mechanism only during local development. In a production environment, sensitive credentials (such as *`SECRET_KEY`*, *`DB_PASSWORD`*, and *`OUTLOOK_CLIENT_SECRET`*) are strictly managed and injected at runtime via HashiCorp Vault to maintain high security standards.

```
# Django Core Settings
DEBUG=True
SECRET_KEY=django-insecure-papillon-secret-key-2025

# Database Configuration (MySQL)
DB_ENGINE=django.db.backends.mysql
DB_NAME=db_name
DB_USER=db_user
DB_PASSWORD=db_pw
DB_HOST=db_host
DB_PORT=db_port

# External Integrations (Microsoft OAuth 2.0)
# Note: In production, these should be fetched from Vault path: papillon/outlook
# OUTLOOK_CLIENT_ID=outlook_client_id
# OUTLOOK_CLIENT_SECRET=outlook_client_secret
```

5.6 Appendix F: Traceability Matrix

Requirement / Design Goal (HLD)	**LLD Component(s) & Logic**	**Verification & Test Evidence (LLD/Unit/Integration)**
Unified AI-driven CTI Platform (Malware, Phishing, IDS)	MalwareDetectionModule, PhishingDetectionModule, NetworkIntrusionModule	**Unit:** Random Forest/XGBoost accuracy validation (min 95%) and ONNX inference consistency checks.
Real-time Situational Awareness (< 2s latency)	WebSocketGateway, DashboardView, Event-driven notification logic	**Integration:** Verification of WebSocket push alerts to UI accordions immediately upon background worker completion.
Asynchronous Heavy Computation (ML & Scanning)	Celery Worker Pool, Message Queue (RabbitMQ/Redis)	**Integration:** API Collector successfully offloads payloads to RabbitMQ without blocking the main request thread.

Security & Isolation (Sandbox/Simulations)	SandboxManager, CALDERA isolated VMs	**Isolation Test:** Verification of strict network isolation with "no host leakage" protocols during execution.
Role-Based Access Control (RBAC)	UserController, CustomUser (Admin, Analyst, ReadOnly)	**Unit/Int:** Unauthorized access attempts to GET /dashboard/ or privileged actions return 401/403 errors.
Multi-Factor Authentication (MFA)	UserController, MFASettingsView, TOTP Base32 generation	**Integration:** Two-phase login flow; verification that the "verified" status in DB is only set after successful OTP entry.
Secure Secret Management (KMS/Vault)	VaultService (Singleton), Fernet (AES-128/256) encryption	**Unit:** VAULT_ENFORCE flag logic; system raises RuntimeError if Vault is unavailable while enforced.
Attack Surface Reconnaissance (Nmap, DNS, SSL)	AttackSurfaceController, Scan Service	**Integration:** Verification of "partial results" handling; system logs errors but continues if one tool (e.g., SSLyze) fails.

Data Normalization (Vendor log unification)	Normalizer Worker (Normalization Service)	**Unit:** Testing the mapping logic of raw JSON logs from disparate vendors into the standard system IDS schema.
Reporting & Audit (PDF Export)	Reporting Service, jsPDF / jsPDF-autotable	**Integration:** handleDownloadPDF correctly aggregates DB results into a formatted, downloadable PDF document.
External API Integration (NIST NVD, MS Graph)	CVEController, OutlookController	**Mock Test:** Successful retrieval of 60-day CVE window and encrypted storage of Microsoft OAuth tokens.
Data Integrity & Persistence (MySQL)	CustomUser, EncryptedData, OutlookAccount models	**Unit:** Use of update_or_create and OneToOneField constraints to prevent duplicate entries and data corruption.
Graceful Degradation (Failback Mechanisms)	Scan Service (Socket fallback), VaultService (.env fallback)	**Unit:** Port scanner falls back to socket if Nmap binary is missing; Vault falls back to env vars in dev mode.

Operational Boundary Conditions (Startup/Shutdown)	Health Check mechanism, Celery signals	**Operational:** "Graceful shutdown" tests ensuring workers finish current jobs before process termination.
Password Security Analytics (ML-based)	PasswordStrengthModule	**Unit:** Model correctly returns strength scores (0, 1, 2) based on entropy and dictionary patterns.

6. References

[1] IBM, "UML - Basics," June 2003. [Online]. Available: http://www.ibm.com/developerworks/rational/library/769.html.

[2] IEEE, "IEEE Citation Reference," September 2009. [Online]. Available: https://m.ieee.org/documents/ieeecitationref.pdf.

[3] Django Software Foundation, "Django Documentation," 2024. [Online]. Available: https://docs.djangoproject.com.

[4] HashiCorp, "Vault Documentation," 2024. [Online]. Available: https://developer.hashicorp.com/vault.

[5] NIST, "National Vulnerability Database API Documentation," 2024. [Online]. Available: https://nvd.nist.gov/developers/vulnerabilities.

[6] Microsoft, "Microsoft Graph API — Mail," 2024. [Online]. Available: https://learn.microsoft.com/en-us/graph/api/resources/mail-api-overview.

[7] Python Cryptography Authority, "cryptography — Hazardous Materials," 2024. [Online]. Available: https://cryptography.io/en/latest/.

[8] ONNX Runtime, "ONNX Runtime Python API," 2024. [Online]. Available: https://onnxruntime.ai/docs/get-started/with-python.html.

[9] sslyze, "sslyze Documentation," 2024. [Online]. Available: https://nabla-c0d3.github.io/sslyze/documentation/.

[10] pyotp, "pyotp — Python TOTP/HOTP Library," 2024. [Online]. Available: https://pyauth.github.io/pyotp/.